



Department of Mathematics and Computer Science
Architecture of Information Systems Research Group

Formal Verification of the Hitchhiking Algorithm Using VerCors

Master's Thesis

Mariam Elsharkawy

Supervisors:
Anton Wijs
Lars Van Den Haak

Eindhoven, August 2026

Abstract

The Hitchhiking algorithm [14] is a BFS-based on-the-fly LTL model-checking algorithm, used to verify Linear-Time Temporal Logic (LTL) formulae. It is a parallel algorithm, suitable for exploiting the computational power of hardware such as graphics processing units. LTL model checking is done by solving the emptiness problem, which is checking whether a directed graph contains an accepting cycle, that is, a cycle through a special kind of node [8]. This thesis uses the VerCors toolset [5] to prove the correctness of this algorithm. To fully prove the correctness of the algorithm, both soundness and completeness need to be proven. Soundness means that whenever the algorithm returns true, a real accepting cycle exists. Completeness means that whenever an accepting cycle exists in the graph, the algorithm returns true (equivalently, whenever the algorithm returns false, no accepting cycle exists). This is first proven at the sequential level, then elevated to the parallel GPU version.

This thesis formalized the hitchhiking algorithm in PVL and used the VerCors toolset to verify its correctness. Soundness was proven at the sequential level, and progress was made toward completeness, including key lemmas required for the full argument. These results lay the foundation for extending verification to the parallel GPU version and for completing the remaining parts of the completeness proof, both of which are left for future work.

Preface

This thesis marks the conclusion of my Master's degree in Computer Science at Eindhoven University of Technology, and presents my work on the formal verification of the Hitchhiking algorithm using the VerCors toolset.

I would first like to thank my supervisor, Anton Wijs, for his guidance throughout this project. His deep knowledge of both the Hitchhiking algorithm itself and the broader field of parallel model checking was invaluable, and his feedback consistently pushed this work to be more precise and more rigorous.

I would also like to thank my co-supervisor, Lars van den Haak, for his close involvement throughout this thesis. He opened my eyes to capabilities of VerCors and formal proof techniques that I did not know existed and would have found very difficult to learn on my own. His guidance and patience in helping me work through the many technical challenges that came with formally verifying this algorithm shaped this work in ways I am deeply grateful for.

Finally, I want to thank my family and friends. The past two years have not been easy, and I would not have made it to this point without their support. To my family, thank you for believing in me and standing by me through every hardship, near and far. To my friends, thank you for your patience, encouragement, and for reminding me that the best is yet to come. This work is as much a product of your support as it is of my own effort.

Mariam Elsharkawy

Contents

Contents	vii
List of Figures	ix
Listings	xi
1 Introduction	1
1.1 Goal and Research Questions	2
1.2 Contributions	2
1.3 Thesis Structure	3
2 Preliminaries	5
2.1 Kripke structure	5
2.2 Path	6
2.3 Linear Temporal Logic	7
2.4 Non-deterministic Büchi Automaton	7
2.5 NBA path	8
2.6 Product of Kripke structure and NBA	8
2.7 Stating the Problem	9
3 The Hitchhiking Algorithm	11
3.1 The Map algorithm	11
3.1.1 Explanation of the Map Algorithm	12
3.1.2 Implementation of the Map algorithm in c++	14
3.1.3 Tracing an example	16
3.1.4 Limitations of the Map algorithm	18
3.2 The Hitchhiking algorithm	19
3.2.1 Explanation of the Hitchhiking algorithm	19
3.2.2 Implementation of the Hitchhiking algorithm in c++	22
4 An introduction for VerCors	25
4.1 Annotations	25
4.2 Permissions	26
4.3 Separating Conjunction	27
4.4 Pure Functions	27
4.5 Termination and Decreases	27
4.6 Loop Invariants	28
4.7 Quantifiers	29
4.8 Verification Lemmas	29
4.9 Sequences	30
4.10 Given and Yields	30
4.11 Practice Exercise: Verifying a BFS Implementation	31
Formal Verification of the Hitchhiking Algorithm Using VerCors	vii

5	The implementation of the Hitchhiking algorithm in PVL	35
5.1	Why PVL	35
5.2	The Graph Representation	35
5.3	The Representation of the Algorithm	37
5.4	Specification Predicates	37
5.5	Function-by-function walkthrough	39
6	Verification of Soundness	41
6.1	Overview of how to prove	41
6.2	Proof sketch	42
6.3	Proof of Condition 1: the shared value is red	42
6.4	Proof of Condition 2: a path from the shared value to the node	43
6.5	Proof of Condition 3: an edge back to the shared value	44
6.6	Wrapping up: the returned witness	44
7	Verification of Completeness	47
7.1	Overview: what the lemmas prove and how they combine	47
7.2	Lemma 2: every node eventually has a real p-value	48
7.3	Lemma 3: the maximum active node bounds every p-value	49
7.4	Lemma 4: the round in which the maximum active node lies in a cycle	50
7.4.1	Status: left for future work	50
7.5	Lemma 5: the maximum active node is never interrupted	51
7.6	Lemma 6: every accepting cycle eventually reopens one of its nodes	52
7.7	Status of the Completeness Proof	53
8	Verification of Termination	55
8.1	Lemma 1: termination of each round	55
8.2	Simple bounded loops	58
8.3	Bounding the number of rounds	58
9	Conclusion and Future Work	61
9.1	An earlier attempt at Lemma 4	61
9.2	Design choices made with future parallel verification in mind	61
9.3	Where this thesis stops	62
9.4	Future work	62
	Bibliography	63
	Appendix	64
A	Pseudocode of the Hitchhiking Algorithm	65
B	The Verification Code of the Hitchhiking Algorithm	67

List of Figures

2.1	An example of a Kripke structure.	6
2.2	An example of a NBA.	8
2.3	An example of a product of a Kripke structure and NBA.	9
3.1	A tracing example of the Map algorithm	16
3.2	After one iteration of <i>update_p</i>	18
3.3	After the first round of <i>update_p</i>	18
3.4	The first interruption case	21
3.5	The second interruption case	22
3.6	The third interruption case	22
6.1	An example graph containing one accepting cycle.	42

Listings

3.1	The implementation of the Map algorithm in C++	14
3.2	The implementation of the Hitchhiking algorithm in C++	22
4.1	A function with a contract	26
4.2	A function with permissions	27
4.3	A pure function	27
4.4	A recursive function with a decreases clause	28
4.5	A loop invariant	29
4.6	A universal quantifier over an array	29
4.7	A verification lemma	30
4.8	A sequence of integers	30
4.9	A method with a given parameter and a yielded result	31
4.10	Verified BFS implementation in VerCors	33
5.1	Checking whether an edge exists	36
5.2	Computing a node's degree	36
5.3	Well-formedness of the CSR graph encoding	37
5.4	Array shape check	37
5.5	Checking that an array is uniformly initialised	37
5.6	Checking a 0/1 flag array	37
5.8	Well-formedness of the algorithm's inputs	38
5.10	Well-formedness of the p-value array	38
5.7	Well-formedness of the algorithm's state arrays	38
5.9	Well-formedness of the open queue	38
5.11	Active states are accepting states	38
5.12	The state invariant	39
6.1	Well-formedness of the function p	43
6.2	Establishing Condition 1	43
6.3	Well-formedness of the ghost-path witnesses	44
6.4	Establishing Condition 2	44
6.5	Establishing Condition 3	44
6.6	Soundness postcondition of processNode	45
6.7	Soundness postcondition of hitchhiking	45
7.1	processNode never re-introduces BOTTOM()	49
7.2	Ghost variable tracking the largest node ever activated	49
7.3	Raising max_active while seeding roots	49
7.4	Raising max_active while processing a child	49
7.5	Bounding p-values and active nodes by the largest node ever activated	50
7.6	A target formalisation of Lemma 4, for future work	51
7.7	max_active_end_round initialised before the round loop	51
7.8	max_active_end_round captured at the end of a round	51
7.9	Lemma 5 as a direct assertion	51
7.10	interruptionInv: the predicate supplying Lemma 5	52
7.11	First interruption site: a first visit to a red child	52

7.12 Second interruption site	52
7.13 Third interruption site	52
8.1 Termination measure for a round	55
8.2 The recursive definition of the running sum	56
8.3 pSeqInv: keeping the running total tied to p	56
8.4 Proving that overwriting one entry shifts the sum by exactly that entry's change	57
8.6 The all-unvisited sequence and its sum	57
8.5 Calling lemma_prefixSum_update alongside a write to p	57
8.7 Establishing the starting point of the sum	58
8.8 Termination measure for the scan inside anyInterrupted	58
8.9 Termination measure for the seeding loop inside processRoots	58
8.10 Termination measure for the post-processing loop inside resetActive	58
8.11 Termination measure for the child loop inside processNode	58
8.13 maxActiveInv: relating the round-bounding ghost variables	59
8.12 Termination measure for the outer round loop	59
8.14 maxActiveInv instantiated as the outer loop's invariant	59
B.1 The complete PVL implementation	67

Chapter 1

Introduction

In 1994, the introduction of the Intel Pentium processor exposed a flaw in its floating point division unit that caused incorrect numerical results for certain rare division operations. As documented by Hoek, the flaw was first brought to public attention when a mathematician observed incorrect results during sophisticated numerical computations and reported the anomaly through Internet newsgroups, after which similar errors were independently confirmed by other users. Although the bug did not cause system failures and appeared only under specific input conditions, it undermined confidence in the correctness of computational results and rapidly escalated into a major crisis [9].

This incident highlights a fundamental limitation of testing-based validation, since rare errors embedded in large state spaces may remain undetected despite extensive testing efforts. From a verification perspective, the Pentium processor flaw illustrates the need to ensure that all executions of a system satisfy a global correctness requirement, namely that every division operation always produces correct results, thereby motivating the use of formal verification techniques such as Linear Temporal Logic model checking, which aim to exhaustively analyze all possible behaviors rather than relying on probabilistic testing [12]. One important LTL model checking algorithm is the Hitchhiking algorithm [14], which is an on-the-fly Linear-Time Temporal Logic (LTL) model-checking algorithm that verifies whether a system satisfies a given property while the state space is being explored.

Traditionally, LTL model checking algorithms are based on Depth-First Search (DFS), as DFS is well suited for detecting cycles in directed graphs. Such algorithms are effective on single-core and small multi-core systems [4]. However, DFS inherently relies on recursion and stack-based exploration, which makes it difficult to parallelize efficiently.

At the same time, modern hardware development increasingly focuses on massive parallelism. In particular, Graphics Processing Units (GPUs) provide thousands of lightweight threads and high memory bandwidth, offering substantial computational power for large-scale verification tasks. To fully exploit this hardware, verification algorithms must be re-designed to avoid inherently sequential exploration strategies such as DFS.

Breadth-First Search (BFS)-based algorithms are more amenable to massive parallelism, and in particular to the massively parallel architecture of GPUs, and are therefore a promising alternative [3, 6, 7, 11]. However, BFS-based approaches to LTL model checking that existed before the Hitchhiking algorithm typically either sacrifice on-the-fly detection or impose restrictions on the length of cycles that can be found [16]. This is problematic, as one of the key advantages of LTL model checking is its ability to detect counterexamples on-the-fly, that is, during state space exploration. On-the-fly detection allows early termination as soon as a violating execution is found, which can significantly reduce verification time and memory usage. This shows the importance of the *Hitchhiking* algorithm.

Just as the Pentium incident demonstrated the risks of undetected implementation errors in hardware, verification algorithms themselves must be trusted to operate correctly. Since the Hitchhiking algorithm targets highly parallel architectures and relies on shared-

memory concurrency, subtle implementation errors such as race conditions may compromise its correctness. This motivates the need for a formal verification of the Hitchhiking algorithm.

To address this problem, the Vercors verification tool is chosen, as it is specifically designed for the formal verification of concurrent and GPU-based programs. It provides support for reasoning about parallel threads, shared memory, and data-race freedom, which are essential aspects of GPU algorithms [1, 15].

1.1 Goal and Research Questions

The goal of this thesis is to formally verify the correctness of the Hitchhiking algorithm using the VerCors toolset. Correctness is established by proving two properties: soundness and completeness. This leads to the following two research questions.

RQ1: Is the Hitchhiking algorithm sound?

Soundness means that whenever the algorithm returns true, a real accepting cycle exists in the graph. Proving soundness ensures that the algorithm never reports a false positive, that is, it never claims a property violation when none exists.

RQ2: Is the Hitchhiking algorithm complete?

Completeness means that whenever an accepting cycle exists in the graph, the algorithm returns true. Proving completeness ensures that the algorithm never reports a false negative, that is, it never misses an actual property violation.

Both research questions are addressed at the sequential level. Proving these properties for the parallel, GPU-based version of the algorithm is left for future work. This thesis provides guidance on how such a proof could be approached, discussed in the context of the algorithm's design in the relevant chapter.

1.2 Contributions

This thesis makes the following contributions.

First, it presents the first formalisation of the sequential version of the Hitchhiking algorithm in PVL, together with an explicit memory safety specification, verified automatically by VerCors.

Second, it presents a complete, machine-checked proof of soundness for the sequential algorithm (RQ1): whenever `hitchhiking` returns true, it also produces a concrete witness proving that an accepting cycle exists, with no unverified assumptions anywhere in the proof.

Third, it presents partial progress towards completeness (RQ2): four of the six lemmas underlying the algorithm's completeness proof, Lemmas 1, 2, 3, and 5, are fully verified with no unverified assumptions, and termination of the algorithm as a whole, not only of each individual round, is proven in full.

Fourth, this thesis documents an attempted proof strategy for Lemma 4, the key remaining lemma, and explains why it did not succeed, so that this avenue does not need to be re-discovered by future work. Lemma 6, which depends directly on Lemma 4, is consequently also left open, and is not itself attempted in this thesis.

Finally, several implementation choices were made specifically to keep a future proof of the parallel, GPU-based version of Hitchhiking within reach, and this thesis provides guidance on how such a proof could be approached.

1.3 Thesis Structure

The remainder of this thesis is structured as follows.

- **Chapter 2:** provides the necessary background information to understand the problem that LTL model checking algorithms try to solve, namely the emptiness-check problem.
- **Chapter 3:** explains the Hitchhiking algorithm in detail, including pseudocode and visual examples, and explains the Map algorithm, which serves as the basis for Hitchhiking.
- **Chapter 4:** justifies the choice of VerCors as the verification tool and provides a tutorial covering the basics of VerCors.
- **Chapter 5:** explains the PVL implementation of the Hitchhiking algorithm.
- **Chapter 6:** presents the proof of soundness.
- **Chapter 7:** presents the progress made toward proving completeness.
- **Chapter 8:** presents the proof that the algorithm terminates.
- **Chapter 9:** discusses an earlier, unsuccessful attempt at proving Lemma 4, the design choices made with future parallel verification in mind, and directions for future work.

Chapter 2

Preliminaries

In this chapter, we introduce the preliminaries needed to understand the *Hitchhiking* algorithm.

At a high level, Linear Temporal Logic (LTL) model checking aims to answer the following question: does a system satisfy a specific property? To answer this question formally, we need three ingredients: a precise model of the system, a precise way of expressing the property we want to check, and a method for checking the property against the model automatically.

We build up these ingredients through a single running example: a simple system that turns on, receives a request, and either grants it immediately or does not. We first model this system as a graph of states and transitions, called a *Kripke structure*, and describe a single execution of the system as a *path* through this graph. We then introduce LTL as a way of expressing properties over such paths. To check an LTL property automatically, its negation is translated into a *Non-deterministic Büchi Automaton* (NBA), which acts as a checker for executions violating the property. Finally, we combine the system and the automaton into a single structure, the *product*, and show that checking whether the system satisfies the property reduces to checking whether this product contains a certain kind of cycle. Following [12], this reduction is known as solving the *emptiness problem*.

The formal definitions in this chapter follow [14] (Section 2, Definitions 1–5), which in turn match, up to minor notational differences, the definitions given in [12].

2.1 Kripke structure

Before verifying any property of a system, we first need a precise mathematical model of the system itself. Consider a simple system that turns on, receives a request, and then either grants the request immediately or does not. We can picture this system as a graph: each node represents a possible state the system can be in, and each edge represents a step the system can take from one state to another. To reason about properties such as “every request is immediately granted,” we additionally need to know, for each state, which relevant facts hold there.

Figure 2.1 shows such a system. State s_0 is where the system starts, before anything has happened. From there, the system turns on and reaches s_1 , where a request occurs. From s_1 , the system either grants the request, moving to s_3 and remaining there forever, or fails to grant it, moving to s_2 , from which it returns to s_1 and the request occurs again.

This kind of model, a graph of states and transitions in which each state is labelled with the facts that hold there, is called a *Kripke structure*. Formally, a Kripke structure over a set of atomic propositions AP is a 4-tuple $K = (S, S_0, \rightarrow, \lambda)$, where

- S is a finite set of states;

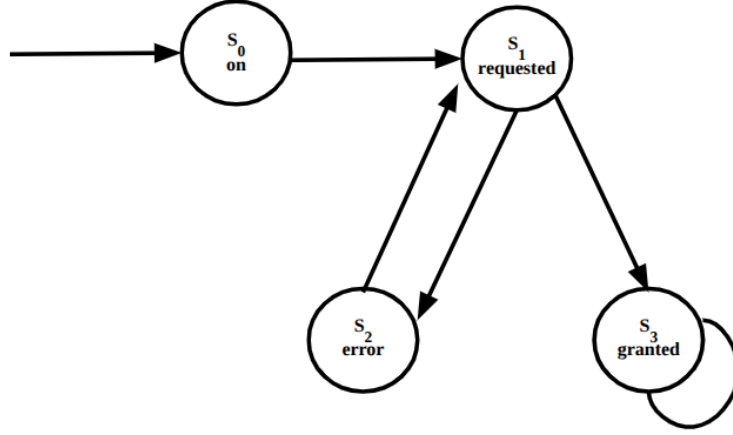


Figure 2.1: An example of a Kripke structure.

- $S_0 \subseteq S$ is the set of initial states;
- $\rightarrow \subseteq S \times S$ is a transition relation, such that $\rightarrow$ is left-total, i.e., $\forall s \in S. \exists s' \in S. (s, s') \in \rightarrow$;
- $\lambda : S \rightarrow 2^{AP}$ is an interpretation (or labelling) function that maps each state to its set of valid atomic propositions.

Returning to our example, let $AP = \{on, requested, granted, error\}$. The system in Figure 2.1 is the Kripke structure $K = (S, S_0, \rightarrow, \lambda)$, where

$$\begin{aligned}
 S &= \{s_0, s_1, s_2, s_3\}, & S_0 &= \{s_0\}, \\
 \rightarrow &= \{(s_0, s_1), (s_1, s_2), (s_2, s_1), (s_1, s_3), (s_3, s_3)\}, \\
 \lambda(s_0) &= \{on\}, & \lambda(s_1) &= \{requested\}, & \lambda(s_2) &= \{error\}, & \lambda(s_3) &= \{granted\}.
 \end{aligned}$$

2.2 Path

A Kripke structure describes everything the system *could* do, but to talk about a single execution, we need to describe one specific run of the system as it unfolds over time. In our example, one possible execution is: the system starts, turns on, receives a request, fails to grant it, receives the request again, fails again, and so on forever, never reaching a grant. This corresponds to following the states $s_0, s_1, s_2, s_1, s_2, \dots$ in Figure 2.1.

Such an infinite sequence of states, where each state actually follows from the previous one via the transition relation, is called a *path*. Formally, given a Kripke structure $K = (S, S_0, \rightarrow, \lambda)$, a path in K is an infinite sequence of states $\pi_K = s_0 s_1 s_2 \dots$, such that $s_0 \in S_0$ and $\forall i \geq 0. s_i \rightarrow s_{i+1}$. Since the transition relation is left-total, paths are guaranteed to be infinite: there is always a next state to move to.

Example. Consider again the Kripke structure defined above in Figure 2.1. One possible path in K is

$$\pi_K = s_0 s_1 s_2 s_1 s_2 s_1 s_2 \dots$$

This is a valid path since $s_0 \in S_0$ and each consecutive pair of states (s_i, s_{i+1}) belongs to the transition relation $\rightarrow$. Note that along this path, a request occurs at s_1 but the very next state, s_2 , is not a grant: the system instead cycles forever between s_1 and s_2 , and a grant never occurs.

2.3 Linear Temporal Logic

We now want a formal language to express properties like “every request is immediately granted.” Following the motivation given in [12] (Section 4.1), a common and natural way to describe system behavior is to describe how it evolves over time, using atomic state properties (facts that hold in a single state) together with logical and temporal connectives that describe in which states these facts must hold. *Linear Temporal Logic* (LTL), proposed by Pnueli in 1977 [12], is one such language. As [12] explains, LTL constrains all possible executions of a system in a uniform way: an LTL formula either holds for *all* paths of a system, or it does not.

LTL combines propositional logic with five temporal connectives. Given a set of atomic propositions AP and $p \in AP$, the syntax of LTL is defined as:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \mathbf{G}\varphi \mid \mathbf{F}\varphi \mid \mathbf{X}\varphi \mid \varphi \mathbf{U} \varphi \mid \varphi \mathbf{W} \varphi.$$

$\mathbf{G}\varphi$ means that φ holds in every reachable state; $\mathbf{F}\varphi$ means that there exists a reachable state where φ holds; $\mathbf{X}\varphi$ means that φ holds in the next state; $\varphi_1 \mathbf{U} \varphi_2$ means that φ_2 will always eventually hold, and until that point φ_1 must hold; and $\varphi_1 \mathbf{W} \varphi_2$ (the *weak until*, sometimes called *unless*) means that either φ_1 holds forever, or if it stops holding, φ_2 must hold at that point.

Given a Kripke structure K and a path $\pi = s_0 s_1 s_2 \dots$ in K , the suffix π^i denotes the path obtained by dropping the first i states of π , i.e., $\pi^i = s_i s_{i+1} s_{i+2} \dots$. Validity of an LTL formula φ for π , denoted $K, \pi \models \varphi$, is then defined inductively:

$$\begin{aligned} K, \pi \models p &\iff p \in \lambda(\pi_0), & K, \pi \models \mathbf{G}\varphi &\iff \forall i. K, \pi^i \models \varphi, \\ K, \pi \models \mathbf{F}\varphi &\iff \exists i. K, \pi^i \models \varphi, & K, \pi \models \mathbf{X}\varphi &\iff K, \pi^1 \models \varphi. \end{aligned}$$

A Kripke structure K satisfies φ , written $K \models \varphi$, if and only if $K, \pi \models \varphi$ for every path π starting in an initial state.

Returning to our example, the property “every request is immediately granted” can be written as

$$\varphi = \mathbf{G}(\text{requested} \rightarrow \mathbf{X}\text{granted}),$$

meaning that at every moment, if a request occurs, then the very next state must be a grant. The path $\pi_K = s_0 s_1 s_2 s_1 s_2 \dots$ from the previous section violates this property: at s_1 , a request occurs, but the very next state, s_2 , is not a grant.

2.4 Non-deterministic Büchi Automaton

A Kripke structure and an LTL formula let us state the system and the property, respectively. But to check the property automatically, we need a mechanism that can scan an execution and decide whether it satisfies the property. Since executions are infinite, this mechanism must be able to make a decision about infinite behavior.

For automata-based LTL model checking, an LTL formula must first be encoded into a type of automaton that is able to accept infinite words; such automata are called ω -automata. Among the possible choices of ω -automata, one commonly used type is the *Non-deterministic Büchi Automaton* (NBA), named after its inventor, Julius Richard Büchi ([12], Section 5.5).

Formally, an NBA is a 5-tuple $\mathcal{B} = (Q, \Sigma, \hookrightarrow, Q_0, Q_F)$, where

- Q is a finite set of states;
- Σ is an alphabet;
- $\hookrightarrow \subseteq Q \times \Sigma \times Q$ is a transition relation;

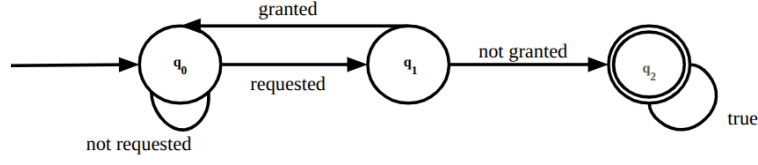


Figure 2.2: An example of a NBA.

- $Q_0 \subseteq Q$ is the set of initial states;
- $Q_F \subseteq Q$ is the set of accepting states.

An NBA has an acceptance condition: a path through an NBA is accepting if and only if it visits a state in Q_F infinitely often.

In automata-based LTL model checking, an NBA is not built directly for the property φ we want to check. A counter-example for φ is a path that violates φ , i.e., that satisfies $\neg\varphi$. Rather than searching for a path that fails to satisfy an automaton for φ , it is more convenient to build an NBA $B_{\neg\varphi}$ directly for the negation $\neg\varphi$, and search for a path that this automaton *does* accept. For every LTL formula φ , it is possible to construct an NBA $B_{\neg\varphi}$ that accepts exactly the paths violating φ .

Figure 2.2 shows an NBA $B_{\neg\varphi}$ for the negation of our running example's property, $\neg\varphi = F(\text{requested} \wedge X\neg\text{granted})$: it accepts exactly those executions in which a request is, at some point, not immediately followed by a grant.

2.5 NBA path

Just as a Kripke structure has paths describing single executions of the system, an NBA has paths describing single runs of the checking machine itself. Formally, given an NBA $B = (Q, \Sigma, \hookrightarrow, Q_0, Q_F)$, a path in B is an infinite sequence of states $\pi_B = q_0q_1q_2\dots$, such that $q_0 \in Q_0$, $\forall i \geq 0. q_i \hookrightarrow q_{i+1}$, $\exists q_i \in \pi_B. q_i \in Q_F$, and $\forall q_i \in \pi_B \cap Q_F. \exists j > i. q_j \in Q_F$. The latter two conditions state precisely that a path is accepting exactly when an infinite number of the states in the path are accepting states.

2.6 Product of Kripke structure and NBA

We now have a model of the system, K , and a checking automaton $B_{\neg\varphi}$ for the bad behavior we want to rule out. To check the property, we run both simultaneously, forming what is called the *product* of K and $B_{\neg\varphi}$.

Formally, given a Kripke structure $K = (S, S_0, \rightarrow, \lambda)$ and an NBA $B = (Q, \Sigma, \hookrightarrow, Q_0, Q_F)$, the product of K and B is an NBA $K \otimes B = (Q_{\otimes}, \Sigma_{\otimes}, \hookrightarrow_{\otimes}, Q_{0,\otimes}, Q_{F,\otimes})$, with

- $Q_{\otimes} = \{\langle s, q \rangle \mid s \in S \wedge q \in Q\}$;
- $\Sigma_{\otimes} = \Sigma$;
- $\hookrightarrow_{\otimes}$ is the smallest relation satisfying the rule:

$$\frac{s \rightarrow s' \quad q \xrightarrow{\phi} q' \quad K, s' \models \phi}{\langle s, q \rangle \xrightarrow{\phi}_{\otimes} \langle s', q' \rangle}$$

- $Q_{0,\otimes} = \{\langle s, q \rangle \in Q_{\otimes} \mid s \in S_0 \wedge \exists q_0 \in Q_0. q_0 \xrightarrow{\phi} q \wedge K, s \models \phi\}$;

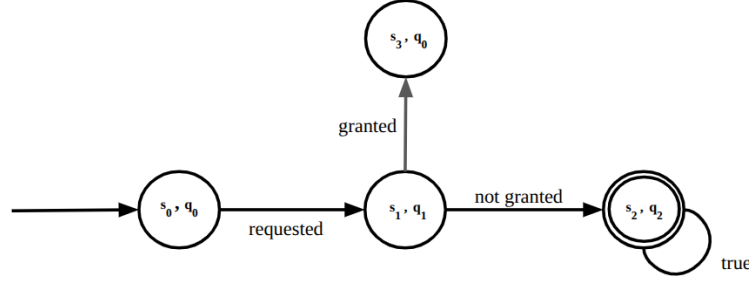


Figure 2.3: An example of a product of a Kripke structure and NBA.

$$Q_{F \otimes} = \{\langle s, q \rangle \in Q_{\otimes} \mid q \in Q_F\}.$$

It holds that K satisfies φ if and only if $K \otimes B_{\neg\varphi}$ has no accepting path, i.e., $\text{paths}(K \otimes B_{\neg\varphi}) = \emptyset$. Searching for a counter-example this way is therefore often referred to as solving the *emptiness problem*: checking whether $K \otimes B_{\neg\varphi}$ is empty.

Figure 2.3 shows the product of the Kripke structure in Figure 2.1 and the NBA in Figure 2.2. Starting from (s_0, q_0) , a request moves the product to (s_1, q_1) . From there, a grant would move it to (s_3, q_0) , but along our example path, the request is instead followed by s_2 , which is not a grant, moving the product to (s_2, q_2) . Since q_2 is accepting and the automaton never leaves it, the states visited while cycling between s_1 and s_2 form an accepting cycle in the product, depicted as the self-loop on (s_2, q_2) in the figure. Thus, the Kripke structure in Figure 2.1 does *not* satisfy $\varphi = G(\text{requested} \rightarrow X \text{ granted})$.

Because $B_{\neg\varphi}$ only accepts a path if it visits an accepting state infinitely often, and the product automaton has finitely many states, an accepting path of $K \otimes B_{\neg\varphi}$ must eventually repeat a state, tracing out a cycle that passes through at least one accepting state. Since a counter-example in a finite-state Kripke structure must be infinite, it necessarily consists of a finite prefix from an initial state to some state, followed by a cycle through that state; this shape is called a *lasso*. Solving the emptiness problem therefore reduces to checking whether the underlying graph of $K \otimes B_{\neg\varphi}$ contains a reachable cycle through an accepting state, i.e., a reachable *accepting cycle*, or equivalently, a reachable accepting lasso.

2.7 Stating the Problem

In this chapter, we introduced the ingredients needed to formally state the problem that the Hitchhiking algorithm solves. We modeled a system as a Kripke structure and described a single execution of the system as a path through it. We introduced LTL as a language for expressing properties over such paths, and showed how an LTL property can be translated into an NBA that accepts exactly the executions violating it. Finally, we combined the system and this automaton into a product, and showed that checking whether the system satisfies the property reduces to checking whether the product contains a reachable accepting cycle, that is, a cycle passing through at least one accepting state. This is the *emptiness-check problem*.

The remainder of this thesis is concerned with algorithms that solve this problem, and with formally verifying their correctness. Before introducing the Hitchhiking algorithm itself, we first explain the *Map algorithm*, because Hitchhiking is built directly on top of it. The Map algorithm is introduced in [6] and later adapted for use on many-core systems such as GPUs [2]. Like Hitchhiking, its goal is to check whether an accepting cycle exists in the product automaton $K \otimes B_{\neg\varphi}$; if such a cycle exists, it represents a counterexample to the LTL property φ . Unlike Hitchhiking, however, the Map algorithm is not on-the-fly: it requires the full state space to be constructed before the check can begin. The Hitchhiking algorithm,

introduced in the next chapter, addresses precisely this limitation of the GPU version of the Map algorithm while retaining its suitability for massively parallel hardware.

Chapter 3

The Hitchhiking Algorithm

The Hitchhiking algorithm is based on the Map algorithm. It is important to distinguish between two versions of Map that appear in the literature:

- The *original* Map algorithm, introduced by Brim et al. [6], is a distributed, multi-core (CPU-based) on-the-fly algorithm. It does not require the full state space to be constructed beforehand.
- The *GPU adaptation* of Map, introduced by Barnat et al. [2], applies the same underlying idea but as a *post-exploration* step: the state space is first fully constructed (at the time, on the CPU, since no GPU-based state space exploration engine existed yet) and then copied into GPU memory, after which Map is applied to the already-complete graph. Because the full state space and transition relation are already resident in memory when Map starts, this version repeatedly scans all transitions of the graph in each iteration, and is consequently not on-the-fly.

These two versions should not be conflated. The properties that motivate the Hitchhiking algorithm, namely that Map is not on-the-fly and that it inspects all transitions in every iteration, hold for the GPU adaptation of Map, and not for the original algorithm of Brim et al. [6]. Osama and Wijs [14] build Hitchhiking specifically as an on-the-fly successor to the GPU adaptation of Map, addressing exactly these two limitations, which only apply to that version. Throughout this thesis, whenever we refer to "the Map algorithm" without further qualification, we mean this GPU adaptation of Map as presented in [14], following the description of Barnat et al. [2], since it is this version that Hitchhiking is designed to improve upon.

3.1 The Map algorithm

In this section we describe the GPU adaptation of Map, since it is this version that forms the basis for the Hitchhiking algorithm and motivates its design. To understand it in depth, I have written out this version of the algorithm sequentially rather than in its parallel, many-core form. In this thesis, I first establish and reason about the sequential version of every algorithm discussed, and elevate it to a parallel or GPU implementation only if time permits; for the work presented here, that elevation was not carried out, and all implementations and results in this thesis are sequential.

We discuss Map here because the Hitchhiking algorithm is designed to resolve the specific limitations of this GPU adaptation. The GPU adaptation of Map originates from the original algorithm of Brim et al. [6] and was adapted for many-core systems such as GPUs by Barnat et al. [2]. At the time that adaptation was developed, no tool existed for constructing state spaces directly on the GPU. As a result, the GPU version of Map could not operate

on-the-fly: the state space first had to be generated on the CPU and only then copied to GPU memory, after which Map was applied as a separate, post-exploration step over the complete, already-known graph.

The goal of the algorithm is to check whether an *accepting cycle* exists in the product automaton $K \otimes B_{\neg\varphi}$. If such a cycle exists, it represents a counterexample to the LTL property φ .

3.1.1 Explanation of the Map Algorithm

We begin by defining the main data structures. For simplicity, we can view the product automaton as a directed graph in which:

- states correspond to *nodes*,
- transitions correspond to *directed edges*,
- initial states are the *roots*, and
- accepting states are special *red nodes*.

The algorithm first collects all states that are reachable from the roots using a breadth-first (BFS-like) traversal. This is implemented by the function `bfs` in Listing 3.1 (lines 8–33), which takes the set of roots `Q0` and the adjacency list `succ` as input. The search starts from the set of root nodes provided as input (loop of line 14) and uses a queue `Q` to perform the breadth-first traversal (loop of line 19). Every node dequeued from `Q` is inserted into the global set `Q` (line 22), which by the end of the traversal holds exactly the set of nodes reachable from the roots. Note that `bfs` only ever populates `Q`; it does not inspect the `is_red` array and therefore does not itself identify which reachable nodes are accepting.

The set of *active* nodes, `A`, which holds the reachable red nodes still being tracked by the algorithm, is instead constructed at the start of `map_function` (loop of line 68): every node `i` already present in `Q` is checked against `is_red`, and if it is red, it is inserted into `A`.

Next, for every node `q`, the algorithm computes its *maximal accepting predecessor*, denoted $p(q)$, which is the largest (according to a predefined total order) red node that can reach `q`. We will refer to the maximal accepting predecessor of a node by the node's `p`-value for short. Each node is given a unique id between 0 and $n - 1$, where n is the number of nodes in the graph. The total order follows from this id.

If for any node `q` it holds that $q = p(q)$, then `q` lies on an accepting cycle, which indicates the presence of a counterexample. In the implementation, the `p`-values are stored in the vector `p` and initialized with $\epsilon = -1$ for every node, together with the vector `old_p` used to detect the fixpoint (line 39). The computation of $p(q)$ is implemented in the function `update_p` (lines 35–63), which takes the graph `g` and the current active set `A` as input.

The computation of $p(q)$ proceeds iteratively. During each iteration, the algorithm examines every edge of every reachable node in the graph and updates $p(q)$ until a fix-point is reached. This iterative propagation is implemented in the loop starting at line 41, where the current `p`-values are copied to `old_p` at the start of the iteration (line 42) and then updated by iterating over all nodes `i` (loop of line 45) and, for each node still in `Q`, over its outgoing edges to each child (loop of line 47). For each edge (i, child) , the algorithm first propagates the maximal predecessor value already known for `i` using the `max` operation (line 49). If `i` is itself an active node, i.e. $i \in A$, its own id is also propagated as a candidate maximal predecessor of `child` (lines 50–52). The iteration stops once no further updates occur in a full pass over all edges, which is detected by comparing `p` to `old_p` (line 57); the resulting fixpoint is returned as `p` (line 62).

After computing the `p`-values, the algorithm checks whether the condition $q = p(q)$ holds for any reachable node. This is implemented in the function `map_function` (lines 65–92), which takes the graph `g` and the `is_red` array as input; note that, unlike `bfs`, it no longer

needs the set of roots, since it only ever operates on the set Q already computed by `bfs`. The function repeatedly calls `update_p` (line 75) as long as A is not empty (line 73). It then iterates over every node $q \in Q$ (loop of line 77) and checks whether $p[q] = q$ (line 78). Because a node's p -value can only ever take on a value from A or ϵ , this condition can in practice only hold for a node that is itself in A ; iterating over all of Q rather than just A does not change the outcome, only the check itself. If the condition holds, an accepting cycle has been detected and the function immediately returns `true` (line 79).

If the condition does not hold for a node q , two clean-up steps are performed instead. First, if $p[q]$ is itself still present in A , it is removed from A (line 83): this reflects that $p[q]$, having propagated as far as it can without closing a cycle at q , can no longer be part of an as-yet-undetected accepting cycle in this round. Second, if q itself is in A but has no outgoing edges, it is also removed from A (line 86); this second check handles the case of a red node without successors. This process repeats, recomputing p from scratch on every round, until either an accepting cycle is found or A becomes empty (line 73), in which case the function returns `false` (line 91).

Listing 3.1 is the implementation of the Map algorithm in C++. It consists of three main functions: `bfs`, `update_p`, and `map_function`.

- The `bfs` function (lines 8–33) explores the graph starting from the root nodes Q_0 and computes the set Q of nodes reachable from them (line 22). It does not itself distinguish red nodes.
- The `update_p` function (lines 35–63) propagates maximal accepting predecessor information through the graph, given the current active set A . It iteratively updates the p -values along the edges of the graph (loop of lines 45–52) until no further changes occur (line 57), thereby computing the maximal accepting predecessor for every reachable node.
- The `map_function` (lines 65–92) first derives the active set A from Q and `is_red` (loop of line 68), then checks whether any node in Q is its own maximal accepting predecessor. It repeatedly invokes `update_p` (line 75) and verifies whether some $q \in Q$ satisfies $p[q] = q$ (line 78). If such a node is found, the function concludes that an accepting cycle exists (line 79); otherwise it prunes A (lines 83–86) and continues until A is empty.

3.1.2 Implementation of the Map algorithm in c++

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4
5  set<int> Q; // set of reachable nodes
6
7  // sequential post-exploration traversal of the graph
8  void bfs(const vector<int>& Q0, const vector<vector<int>>& succ) {
9
10     int n = succ.size();
11     vector<bool> visited(n, false);
12     queue<int> O; // set of open nodes
13
14     for (int root : Q0) {
15         if (!visited[root]) {
16             visited[root] = true;
17             O.push(root);
18
19             while (!O.empty()) {
20                 int node = O.front();
21                 O.pop();
22                 Q.insert(node);
23
24                 for (int child : succ[node]) {
25                     if (!visited[child]) {
26                         visited[child] = true;
27                         O.push(child);
28                     }
29                 }
30             }
31         }
32     }
33 }
34
35 vector<int> update_p(const vector<vector<int>>& g, const set<int>& A) {
36
37     // the value epsilon in the paper is represented as -1 here
38     vector<int> p(g.size(), -1);
39     vector<int> old_p(g.size(), -1);
40
41     while (true) {
42         old_p = p; // line 19 in the pseudocode
43
44         // The two loops iterates over all the nodes in the graph (line 20)
45         for (int i = 0; i < g.size(); i++) {
46             if (Q.find(i) == Q.end()) continue;
47             for (auto child : g[i]) {
48                 if (Q.find(child) == Q.end()) continue;
49                 p[child] = max(p[child], p[i]);
50                 if (A.find(i) != A.end()) {
51                     p[child] = max(p[child], i);
52                 }
53             }
54         }
55
56         if (p == old_p) { // line 22 in the pseudocode
57             break;
58         }
59     }
60
61     return p;
62 }
63
64 bool map_function(const vector<vector<int>>& g, const vector<bool>& is_red) {

```

```

66
67     set<int> A; // set of active nodes
68     for (auto i : Q) { // line 9 in the pseudocode
69         if (is_red[i])
70             A.insert(i);
71     }
72
73     while (!A.empty()) {
74         vector<int> p;
75         p = update_p(g, A);
76
77         for (auto q : Q) { // line 12 in the pseudocode
78             if (p[q] == q) {
79                 return true; // counter-example found
80             }
81             else { // line 14 in the pseudocode
82                 if (A.find(p[q]) != A.end())
83                     A.erase(p[q]);
84
85                 if (A.find(q) != A.end() && g[q].empty())
86                     A.erase(q);
87             }
88         }
89     }
90
91     return false; // no counter-example
92 }
93
94 int main() {
95     int q0; // number of roots
96     cin >> q0;
97     vector<int> Q0(q0);
98     for (int i = 0; i < q0; i++) {
99         cin >> Q0[i];
100     }
101
102     int n; // number of nodes
103     cin >> n;
104     int m; // number of edges
105     cin >> m;
106     vector<vector<int>> succ(n); // adjacency list representation of the graph
107
108     for (int i = 0; i < m; i++) {
109         int u, v;
110         cin >> u >> v;
111         succ[u].push_back(v);
112     }
113
114     int num_red_nodes; // number of red nodes
115     cin >> num_red_nodes;
116     vector<bool> is_red(n, false);
117     for (int i = 0; i < num_red_nodes; i++) {
118         int red_node;
119         cin >> red_node;
120         is_red[red_node] = true;
121     }
122
123     // Perform BFS from each root
124     bfs(Q0, succ);
125
126     if (map_function(succ, is_red))
127         cout << "Detected an accepting cycle, counter example found!" << endl;
128     else
129         cout << "No accepting cycle detected, no counter example found!" << endl;
130 }
131 }

```

Listing 3.1: The implementation of the Map algorithm in C++

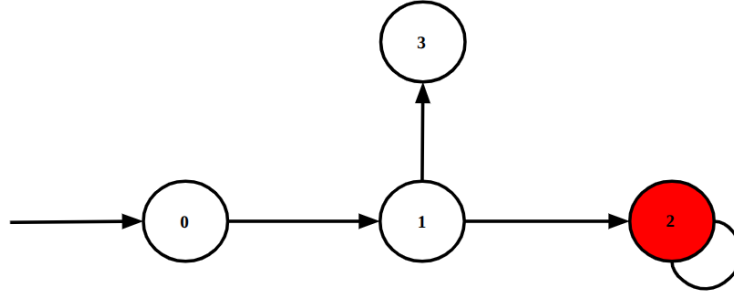


Figure 3.1: A tracing example of the Map algorithm

3.1.3 Tracing an example

In this section, we trace the example of the product of the Kripke structure and the NBA presented in the previous chapter, but using the notation introduced in this chapter. Under this notation, nodes are labeled with numbers that also represent their priorities. Hence, node 2 has a higher priority than node 1, which follows the predefined total order discussed earlier. Accepting states are highlighted in red. Figure 3.1 illustrates the same example with these modifications. The following presents the trace of this graph using the proposed code.

Step 1: BFS exploration

The algorithm first performs a breadth-first search starting from the root node (Listing 3.1, lines 8–33).

$$0 \rightarrow 1 \rightarrow 2, 3$$

Thus, the set of reachable nodes becomes

$$Q = \{0, 1, 2, 3\}$$

Step 2: Construction of the active set A

At the start of `map_function` (line 68), every node of Q is checked against `is_red`. Since node 2 is the only accepting node in Q ,

$$A = \{2\}$$

Step 3: Initialization of $p(q)$

Next, inside `update_p`, the maximal accepting predecessor values are initialized (line 39):

$$p(q) = -1 \quad \forall q \in \{0, 1, 2, 3\}$$

Thus,

$$p = [-1, -1, -1, -1]$$

Step 4: First iteration of propagation

The algorithm propagates maximal accepting predecessors along the edges of the graph (lines 45–52), processing nodes in order 0, 1, 2, 3 and, for each, its outgoing edges.

- Edge $0 \rightarrow 1$

Node 0 is not in A and $p[0] = -1$, therefore no update occurs.

$$p = [-1, -1, -1, -1]$$

- Edge $1 \rightarrow 2$

Node 1 is not in A and $p[1] = -1$, so no update occurs.

$$p = [-1, -1, -1, -1]$$

- Edge $1 \rightarrow 3$

Node 1 is not in A and $p[1] = -1$, therefore no update occurs.

$$p = [-1, -1, -1, -1]$$

- Edge $2 \rightarrow 2$

Node 2 is in A . Its own id is therefore propagated to itself (lines 50–52).

$$p[2] = \max(-1, 2) = 2$$

$$p = [-1, -1, 2, -1]$$

Since the values of p changed, another iteration is performed.

Step 5: Second iteration

Processing the edges again:

- Edge $0 \rightarrow 1$

No change occurs.

- Edge $1 \rightarrow 2$

$$p[2] = \max(2, -1) = 2$$

No change occurs.

- Edge $1 \rightarrow 3$

$$p[3] = \max(-1, -1) = -1$$

No change occurs.

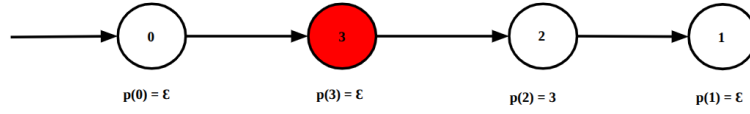
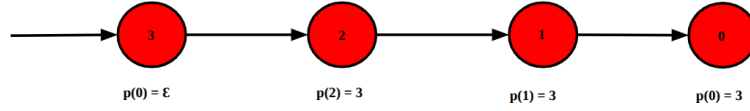
- Edge $2 \rightarrow 2$

$$p[2] = \max(2, 2) = 2$$

No change occurs.

Since no further updates occur, the fixpoint is reached (line 57):

$$p = [-1, -1, 2, -1]$$


 Figure 3.2: After one iteration of *update_p*

 Figure 3.3: After the first round of *update_p*

Step 6: Cycle detection

Back in *map_function*, the algorithm now checks, for every $q \in Q$, whether

$$p(q) = q$$

Iterating over $Q = \{0, 1, 2, 3\}$ in order (line 77), the first node for which this holds is 2:

$$p(2) = 2$$

which means that node 2 is its own maximal accepting predecessor.

Conclusion

Since an accepting node can reach itself, the algorithm detects an accepting cycle (line 79). Therefore, the Map algorithm reports the existence of a counterexample.

3.1.4 Limitations of the Map algorithm

1. In each iteration, *update_p* checks *all* edges of the graph (lines 45–52), even though in the first iteration only the edges leaving an active node in A can actually cause an update, since every other node's p-value is still ϵ . This is illustrated in Figure 3.2: it shows the p-value of every node after one iteration of *update_p* on a chain $0 \rightarrow 3 \rightarrow 2 \rightarrow 1$, with node 3 red. Only node 2, the successor of the red node 3, receives an updated p-value; the p-value of every other node remains ϵ , yet the loop of lines 45–47 still inspected every edge of the graph to establish this. In subsequent iterations, the effect of the red node 3 propagates further along the chain, but each of those iterations again re-scans every edge rather than only the ones still capable of changing, resulting in unnecessary computation and reduced efficiency.
2. The algorithm does not distinguish between searches that have *finished* propagating and those that have merely been *interrupted*. This is illustrated in Figure 3.3, which shows the same chain after a full round of *update_p* has reached its fixpoint: every node's p-value now points to node 3, the highest-priority red node. Back in *map_function*, this causes node 3 to be erased from A at line 83 for every q whose $p[q] = 3$, since none of them satisfy $p[q] = q$. If A is not yet empty, the loop of line 73 starts a new round, calling *update_p* again, which re-initializes *every* node's p-value

back to ϵ (line 39) rather than only reopening the search from the node that was actually interrupted. In some cases, this leads to redundant recomputation of propagations that had, in fact, already run to completion.

This motivates the invention of the Hitchhiking algorithm.

3.2 The Hitchhiking algorithm

The Hitchhiking algorithm is introduced in [14]. It addresses the shortcomings of the GPU version of the Map algorithm by offering a more efficient and parallel-friendly method for detecting accepting cycles. In contrast to Map, which performs cycle detection after the state space has been fully explored, Hitchhiking detects accepting cycles on the fly, that is, during state-space exploration. Moreover, Hitchhiking explicitly tracks interruptions in accepting-state propagations.

3.2.1 Explanation of the Hitchhiking algorithm

We begin by defining the main data structures. The algorithm uses the same underlying graph representation as the Map algorithm (nodes, directed edges, roots, and red nodes), together with three sets and a labeling vector that directly mirror the notation O , A , F , and p used in Algorithm 2 of [14]:

- O : the open set, containing the nodes that are scheduled to be explored.
- A : the set of currently active red nodes, i.e. the nodes for which a cycle search is ongoing.
- F : the set of red nodes whose searches were *interrupted* by other, higher-priority red nodes.
- p : a vector mapping each node to its current maximal accepting predecessor, or to one of the special values `bottom` and `epsilon`.

Listing 3.2 presents the C++ implementation of the Hitchhiking algorithm. In contrast to the Map algorithm, Hitchhiking does not require the entire state space to be explored beforehand. Instead, it interleaves exploration and verification, performing on-the-fly propagation of red predecessors and immediate detection of cycles.

Before exploration starts, the input node identifiers are normalized: the helper function `get_id` (lines 15–19) assigns every node a fresh, contiguous id, in the order in which it is first encountered while reading the roots, the edges, and the red nodes from the input (lines 25–50). All subsequent comparisons between node identifiers, and in particular the total order used to decide which of two accepting nodes has higher priority, are made with respect to these normalized ids. Membership in the sets O , A , and F is tested throughout the code using the helper function `in` (lines 8–10).

The two special values used in the labeling vector p are defined as constants `epsilon` and `bottom` (line 6). A node labeled `bottom` indicates that the node has not been reached before. A node labeled `epsilon` indicates that it has been encountered, but no accepting predecessor has yet been assigned to it. The vector p is initialized with `bottom` for every node (line 55).

Initially, every root node is added to the open set O (line 58). If a root node is accepting, it is also inserted into the active set A and its own label is set to itself (lines 59–62); otherwise its label is set to `epsilon` (line 64).

The algorithm then proceeds in rounds, implemented by the outer loop starting at line 68, which continues as long as O is not empty. Each round consists of two phases: an exploration

phase, implemented by the inner loop of lines 71–105, and, if needed, a post-processing phase, implemented in lines 107–127.

During the exploration phase, nodes are repeatedly removed from O one at a time (lines 72–73) and processed, and any successor whose label improves as a result is reinserted into O (line 101) to be processed later in the same phase. This loop therefore keeps draining O , including nodes added to it while the loop is running, until no further updates are pending and O is empty, which corresponds to propagation having reached a fixpoint for the current round.

For each node $node$ popped from O , the algorithm retrieves its current label $\alpha \leftarrow p(node)$ (line 75) and iterates over its successors $child \in succ(node)$ (line 77).

If a successor satisfies $\alpha = child$, this indicates that an accepting cycle has been found. In that case, the algorithm immediately terminates and reports a counterexample (lines 78–80). This allows Hitchhiking to detect cycles as soon as they appear during exploration.

When a successor node is encountered for the first time and it is accepting, i.e. $p(child) = \text{bottom}$ and $\text{is_red}[child]$ holds, the node is inserted into the active set A (lines 83–84). If the newly discovered red node has a higher identifier than the current maximal predecessor α , it becomes the new predecessor (lines 86). Otherwise, the node is added to the set F (line 88).

After this step, the algorithm updates the label of the child node by propagating the maximal predecessor. The previous label $\beta = p(child)$ is stored (line 91), and the new label is computed as $p(child) = \max(\alpha, \beta)$ (line 92).

There are three cases where a node is added to the set F :

- **First interruption case.** If a new red child, i.e. a node whose label is still bottom, is discovered with an identifier smaller than the current maximal predecessor α , then its exploration is postponed since a higher-priority red node dominates it. This situation is implemented in lines 83–88.

As shown in Figure 3.4, node 4 is being processed with $\alpha = p(4) = 5$, and its child, node 3, is being visited for the first time ($p(3) = \perp$). Because 3 is not greater than $\alpha = 5$, node 3 does not become the new predecessor; instead, it is added to F , since its label is about to be overwritten to 5 rather than kept as 3 when the propagation step of lines 91–92 is executed next, even though node 3 lies on the accepting cycle $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 4$. Node 3 must therefore be reprocessed later.

- **Second interruption case.** If the propagation improves the label of a child that is already scheduled for exploration, the previous propagation becomes obsolete and the corresponding predecessor is marked as interrupted. This case is implemented in line 95.

As shown in Figure 3.5, assume that nodes 2, 4, and 5 are processed in that order. When node 2 is processed, its child 4 receives label $p(4) = 2$ and is added to O . When node 5 is later processed with $\alpha = p(5) = 5$, its child is again node 4, now with $\beta = p(4) = 2$; since $\alpha > \beta$, $\beta > \text{epsilon}$, and node 4 is still in O , node 2 is added to F , and $p(4)$ is updated to 5. Node 2 must therefore be reopened and reprocessed later, since it is part of an accepting cycle.

- **Third interruption case.** If a child already has a higher maximal predecessor than the one currently being propagated, and this child does not belong to the active set A , then the current propagation attempt is interrupted by this stronger predecessor. This case is implemented in line 98.

As shown in Figure 3.6, assume that nodes 5, 4, and 2 are processed in that order. By the time node 2 is processed with $\alpha = p(2) = 2$, its child 4 already carries the higher label $\beta = p(4) = 5$, set earlier by node 5's search. Since node 4 is not red and hence not in A , node 2 is added to F and will be reprocessed later, since it is part of an accepting cycle.

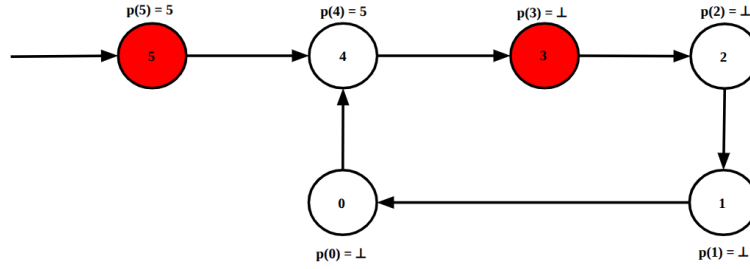


Figure 3.4: The first interruption case

If the propagation produces a stronger predecessor for a child node, that child is added back to O so that the improved information can be further propagated through the graph (lines 101).

Once the exploration phase has drained O , post-processing of the round begins (line 107) if the set F is not empty. In the loop of lines 108–126, every node is inspected. An active node that has been interrupted and whose label does not yet reference itself is reinserted into O so its search can resume (lines 112); an active node that was not interrupted is instead removed from A , as it cannot be part of an as-yet-undetected accepting cycle in this round (line 115). The set F is then cleared (lines 119). Finally, p is reset for the next round: nodes still in A start referencing themselves again, and all other nodes revert to ϵ (lines 125).

This process continues until either an accepting cycle is detected, or the open set O becomes empty at the start of a round, in which case the algorithm reports that no accepting cycle exists (line 130).

Listing 3.2 implements Hitchhiking in a single main function, structured in three parts.

- *Initialization* (lines 52–66) creates the sets O , A , F and the vector p , and seeds O and A from the roots.
- *Round exploration* (lines 71–105) drains O , propagating labels along edges, detecting cycles at line 78, and populating F whenever a propagation is interrupted (lines 88, 95, and 98).
- *Round post-processing* (lines 107–127) reactivates interrupted searches, deactivates finished ones, and resets p for the nodes still active, before the next round begins at line 68.

This approach resolves the limitations of the Map algorithm by avoiding the need for global fixpoint computation over the entire reachable state space. Instead, Hitchhiking performs localized propagation and dynamically suspends and resumes searches when stronger accepting predecessors appear. As a result, it enables early cycle detection and supports efficient on-the-fly verification.

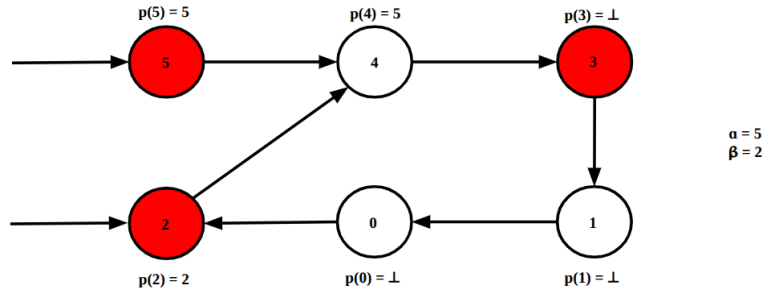


Figure 3.5: The second interruption case

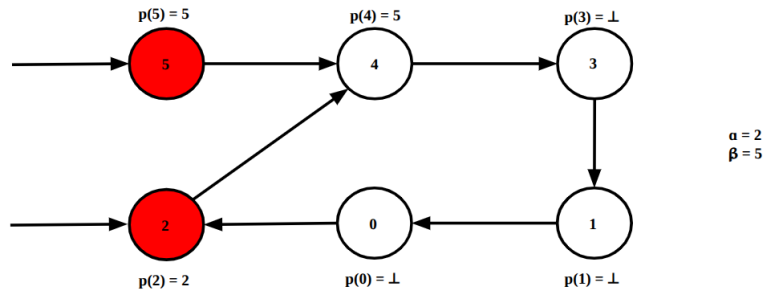


Figure 3.6: The third interruption case

3.2.2 Implementation of the Hitchhiking algorithm in c++

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 const int epsilon = -1;
6 const int bottom = -2;
7
8 bool in(int element, const set<int>& s) {
9     return s.find(element) != s.end();
10 }
11
12 unordered_map<int,int> id;
13 int nxt = 0;
14
15 int get_id(int x) {
16     if (!id.count(x))
17         id[x] = nxt++;
18     return id[x];
19 };
20
21 int main() {
22
23     int num_roots;
24     cin >> num_roots;
25     vector<int> Q0(num_roots);
26     for (int i = 0; i < num_roots; i++) {
27         cin >> Q0[i];
28         Q0[i] = get_id(Q0[i]);
29     }
30
31     int num_nodes, num_edges;
32     cin >> num_nodes >> num_edges;
    
```

```

33 vector<vector<int>> succ(num_nodes);
34 for (int i = 0; i < num_edges; i++) {
35     int u, v;
36     cin >> u >> v;
37     u = get_id(u);
38     v = get_id(v);
39     succ[u].push_back(v);
40 }
41
42 vector<bool> is_red(num_nodes, false);
43 int num_red_nodes;
44 cin >> num_red_nodes;
45 for (int i = 0; i < num_red_nodes; i++) {
46     int red_node;
47     cin >> red_node;
48     red_node = get_id(red_node);
49     is_red[red_node] = true;
50 }
51
52 set<int> O;
53 set<int> A;
54 set<int> F;
55 vector<int> p(num_nodes, bottom);
56
57 for (int root : Q0) {
58     O.insert(root);
59     if (is_red
60         [root]) {
61         A.insert(root);
62         p[root] = root;
63     } else {
64         p[root] = epsilon;
65     }
66 }
67
68 while (!O.empty()) {
69
70     while (!O.empty()) {
71         int node = *O.begin();
72         O.erase(node);
73
74         int alpha = p[node];
75
76         for (int child : succ[node]) {
77             if (alpha == child) {
78                 cout << "Detected an accepting cycle, counter example found!" <<
79                     endl;
80                 return 0;
81             }
82
83             if (p[child] == bottom && is_red[child]) {
84                 A.insert(child);
85                 if (child > alpha)
86                     alpha = child;
87             } else
88                 F.insert(child); //first interruption case
89         }
90
91         int beta = p[child];
92         p[child] = max(alpha, beta);
93
94         if (alpha > beta && beta > epsilon && in(child, O))
95             F.insert(beta); // second interruption case
96
97         if (beta > alpha && alpha > epsilon && !in(child, A))
98             F.insert(alpha); // third interruption case

```

```
99
100         if (alpha > beta)
101             O.insert(child);
102     }
103
104
105 }
106
107 if (!F.empty()) {
108     for (int node = 0; node < num_nodes; node++) {
109         if (in(node, A)) {
110             if (in(node, F) && p[node] != node) {
111                 O.insert(node);
112             }
113             else {
114                 A.erase(node);
115             }
116
117             if (in(node, F)) {
118                 F.erase(node);
119             }
120         }
121         if (in(node, A))
122             p[node] = node;
123         else
124             p[node] = epsilon;
125     }
126 }
127 }
128 }
129
130 cout << "No accepting cycle detected, no counter example found!" << endl;
131
132 }
```

Listing 3.2: The implementation of the Hitchhiking algorithm in C++

Chapter 4

An introduction for VerCors

VerCors is introduced in this chapter since it is chosen to verify the Hitchhiking algorithm. VerCors is also used to verify a model checking algorithm in [10]. VerCors is a deductive verification framework for concurrent and parallel programs that supports formal analysis of software written in languages such as Java, C and OpenCL. It relies on permission-based separation logic to express and verify properties including data-race freedom, memory safety, and functional correctness at the level of concrete implementations. The tool works by translating annotated source code into an intermediate verification language and applying back-end solvers to determine whether the program meets its formal specifications. VerCors is specifically built to handle heterogeneous and highly parallel programs, making it well-suited for the verification of GPU kernels and other complex parallel software systems. This chapter is mostly based on the official VerCors tutorial¹ and the background section of [15].

4.1 Annotations

The keyword `requires` specifies a precondition, while the keyword `ensures` specifies a postcondition. The result of a function is denoted by `\result`. The statement context `P` is an abbreviation for `requires P` and `ensures P`. Additionally, the annotation `context_``everywhere P` indicates that the condition `P` holds throughout the execution of the function; it is automatically added as an annotation wherever required.

Listing 4.1 demonstrates the use of contract annotations in a simple function that increments its argument. The precondition `requires x > 0` restricts valid inputs to positive values. The postconditions then specify the guarantees provided by the function: the result is equal to `x + 1` and, consequently, greater than the original input.

¹<https://vercors.ewi.utwente.nl/>

```
1 requires x > 0;  
2 ensures \result == x + 1;  
3 ensures \result > x;  
4  
5 int increment(int x) {  
6     return x + 1;  
7 }
```

Listing 4.1: A function with a contract

4.2 Permissions

VerCors uses a permission-based separation logic to reason about safe access to shared memory in concurrent and parallel programs. Permissions specify whether a thread may read from or write to a memory location, allowing VerCors to detect data races and ensure memory safety. A permission is written as $\text{Perm}(l, r)$, where l denotes a heap location and r is a fraction between 0 and 1. A full permission ($1 \setminus 1$), also denoted by `write`, grants write access, while fractional permissions grant read-only access.

The total permission for any memory location across all threads must not exceed 1, ensuring that at most one thread can write to a location and that reading cannot occur concurrently with writing. Permissions can be split and combined using the separating conjunction, enabling modular reasoning about different parts of the heap.

The postcondition uses the `\old` construct to relate the updated value to the value at function entry, expressing functional correctness in addition to memory safety.

Listing 4.2 demonstrates the use of permission annotations for a function that modifies an element of an array. The context conditions specify the assumptions under which the function may be executed: the array reference `xs` must not be `null`, the array must contain at least one element, and the function must hold write permission for the location `xs[0]`. This permission allows the function to safely modify the first element of the array. The postcondition then guarantees that after execution, the value stored at `xs[0]` is equal to 0.

```

1 context xs != null && xs.length >= 1;
2 context Perm(xs[0], write);
3 ensures xs[0] == 0;
4
5 void resetFirst(int[] xs) {
6     xs[0] = 0;
7 }

```

Listing 4.2: A function with permissions

4.3 Separating Conjunction

The separating conjunction (******) combines permission predicates in VerCors specifications. A formula $P \text{ ** } Q$ states that both P and Q hold, with the added requirement that the ownership resources in P and Q are disjoint: together they must not exceed permission 1 for any location. This makes ****** stronger than the ordinary logical conjunction $\wedge$. For example, from $\text{Perm}(x.f, 1) \text{ ** } \text{Perm}(y.f, 1)$ one can conclude $x \neq y$, since a single field cannot hold two full write permissions at once.

A permission $\text{Perm}(o.f, q)$ can be split into $\text{Perm}(o.f, q \setminus 2) \text{ ** } \text{Perm}(o.f, q \setminus 2)$, and conversely two permissions for the same location can be merged: $\text{Perm}(o.f, q_1) \text{ ** } \text{Perm}(o.f, q_2)$ equals $\text{Perm}(o.f, q_1 + q_2)$.

4.4 Pure Functions

Pure functions in VerCors are functions that have no side effects and do not modify the heap. As a result, they do not require permissions and can be safely used inside specifications such as preconditions, postconditions, assertions, and loop invariants. A pure function is declared using the keyword **pure** and must consist of a single expression or a body that can be translated into a single expression.

Because pure functions do not alter program state, their bodies are available in the verification context and can be used directly when reasoning about specifications. This is unlike non-pure methods, whose behaviour must instead be captured explicitly through a contract (pre- and postconditions), since their internal behaviour is not exposed to the verifier when used in specifications.

```

1 pure int double(int x) = 2 * x;

```

Listing 4.3: A pure function

Listing 4.3 defines a pure function `double` that returns twice its argument; because it is declared **pure**, its body can be used directly wherever `double(x)` appears in a specification, without needing a separate contract.

4.5 Termination and Decreases

VerCors can verify termination of functions and loops using the **decreases** annotation. A **decreases** clause specifies an integer expression that must remain non-negative and strictly decrease with every recursive call or loop iteration. If these conditions are satisfied, VerCors proves that the execution eventually terminates.

For recursive functions, the **decreases** clause is typically added to the function contract and is checked for every recursive call. For loops, a **decreases** clause may be added to the loop contract to prove loop termination. When termination is specified at the function level, VerCors also checks that all loops and function calls within the function body terminate.

```
1 requires n >= 0;  
2 decreases n;  
3 void count_down(int n) {  
4     if (n > 0) {  
5         count_down(n - 1);  
6     }  
7 }
```

Listing 4.4: A recursive function with a decreases clause

Listing 4.4 shows the decreases n clause, which tells VerCors that n strictly decreases and stays non-negative on every recursive call, proving that `count_down` always terminates.

4.6 Loop Invariants

Loop invariants in VerCors are annotations that describe properties which must hold before entering a loop, be preserved by every iteration, and may be assumed once the loop terminates. They are specified using the `loop_invariant` keyword.

Loop invariants typically express constraints on loop variables, relationships between program variables, and functional correctness properties that evolve during loop execution. In addition, loop invariants may include permissions, ensuring that the necessary access rights to memory locations are maintained throughout the loop.

To verify a loop, VerCors checks that all loop invariants hold initially, are preserved by the loop body, and, together with the loop termination condition, imply the desired post-conditions after the loop completes. Loop invariants are often combined with a decreases clause to prove loop termination.

Listing 4.5 uses `loop_invariant 0 <= i && i <= n` to state that i always stays within bounds during the loop, which VerCors checks before the loop starts, after every iteration, and can use to reason about the code that follows the loop.


```

1 context n >= 0;
2 void count_up(int n) {
3     loop_invariant 0 <= i && i <= n;
4     for (int i = 0; i < n; i = i + 1) {
5     }
6 }

```

Listing 4.5: A loop invariant

4.7 Quantifiers

VerCors supports universal and existential quantification from first-order logic, allowing specifications to express properties over ranges of values. Universal quantifiers are written as $(\forall T\ x; P)$ and existential quantifiers as $(\exists T\ x; P)$, where P is a logical expression. Often, a restricted form $(\forall T\ x; D; P)$ is used as shorthand for $(\forall T\ x; D \Rightarrow P)$, enabling concise specification of array bounds and similar conditions.

```

1 context xs != null;
2 context (\forall forall* int i; 0 <= i && i < xs.length; Perm(xs[i], read));
3 context (\forall forall int i; 0 <= i && i < xs.length; xs[i] >= 0);
4 void check_non_negative(int[] xs) {
5 }

```

Listing 4.6: A universal quantifier over an array

Listing 4.6 uses a universal quantifier `\forall int i; 0 <= i && i < xs.length; xs[i] >= 0` to state that every element of `xs` is non-negative, expressing a property over the whole array in a single specification line.

4.8 Verification Lemmas

A verification lemma in VerCors is a helper function whose sole purpose is to prove a property needed during verification. Semantically, a lemma is an ordinary function with a contract, but it is not intended to contribute to the program's functional behavior. Instead, it is used to establish facts that can be reused elsewhere in specifications or proofs.

Verification lemmas are often written in a recursive style to mimic mathematical induction. To ensure soundness, lemmas must be terminating; therefore, a decreases clause is required for recursive lemmas. Without a proven termination argument, VerCors may accept unsound proofs.

Listing 4.7 illustrates an example of a verification lemma. In this example, `sum(n)` denotes the sum of the first n natural numbers, typically defined recursively by `sum(0) = 0` and `sum(n) = n + sum(n-1)` for $n > 0$. The lemma proves that `sum(n) ≥ 0` for all $n ≥ 0$ by recursively calling itself on $n - 1$, which mirrors an inductive proof.

```
1 requires n >= 0;  
2 ensures sum(n) >= 0;  
3 decreases n;  
4 void lemma_sum_nonnegative(int n) {  
5     if (n > 0) {  
6         lemma_sum_nonnegative(n - 1);  
7     }  
8 }
```

Listing 4.7: A verification lemma

4.9 Sequences

The type `seq<T>` represents an ordered, immutable collection of values of type `T`, used inside specifications to reason about data abstractly (for example, the contents of an array). A sequence can be constructed explicitly, and its length is written `|s|`.

```
1 void seq_example() {  
2     seq<int> s = seq<int> { 1, 2, 3 };  
3     assert |s| == 3;  
4     assert s[0] == 1;  
5 }
```

Listing 4.8: A sequence of integers

Listing 4.8 constructs a sequence `s` containing the integers 1, 2, and 3 inside a method body, and asserts that its length is 3 and that its first element, accessed with `s[0]`, is 1.

4.10 Given and Yields

The `given` and `yields` keywords extend a method’s contract with extra ghost parameters and ghost return values, which exist only for specification purposes. A `given` parameter can be referenced in the contract like a normal parameter; a `yields` value can be referenced in the postcondition and is retrieved by the caller after the call. Both are supplied at the call site, in a specification comment placed between the method call and its semicolon.

Listing 4.9 shows `add_extra` taking a ghost input `extra` via `given` and producing a ghost output `total` via `yields`; the caller supplies `extra` at the call site and retrieves `total` into the ghost variable `result`.

```

1 given int extra;
2 yields int total;
3 requires extra >= 0;
4 ensures total == x + extra;
5 void add_extra(int x) {
6     ghost total = x + extra;
7 }
8
9 void caller() {
10     ghost int res;
11     add_extra(5) given {extra=3} yields {res=total};
12 }

```

Listing 4.9: A method with a given parameter and a yielded result

4.11 Practice Exercise: Verifying a BFS Implementation

In order to gain hands-on experience with VerCors and become familiar with its permission-based reasoning style, an exercise was conducted in which a Breadth-First Search (BFS) algorithm was specified and verified.

The goal of this exercise was not to establish deep functional correctness of graph reachability, but rather to practice writing specifications, managing permissions, and constructing loop invariants. The implementation uses the Compressed Sparse Row (CSR) format [13] of a graph via the arrays *R* and *C*, where *R*[*v*] and *R*[*v*+1] delimit the range of neighbors of vertex *v* in *C*.

The verification focuses on memory safety and race-free access by explicitly specifying read permissions for the graph structure and write permissions for the distance array. The algorithm iteratively propagates distance values from the source vertex *s*, mimicking a level-synchronous BFS.

Several loop invariants were introduced to ensure:

- safe access to array bounds,
- preservation of control variables such as *done*,
- sound iteration over adjacency ranges.

This exercise helped in understanding how VerCors requires explicit reasoning about heap permissions and how loop invariants must capture both data safety and control-flow properties.

Listing 4.10 presents the verified BFS implementation.

The following is a detailed explanation of how the BFS implementation is verified in VerCors, focusing on memory safety and correct permission usage.

Global Permissions

- `context_everywhere R != null ** ... Perm(R[i], read)` Ensures that the adjacency pointer array *R* exists and that every element may be read. Since BFS never modifies *R*, read permission is sufficient.
- `context_everywhere C != null ** ... Perm(C[i], read)` Grants read access to the adjacency list array *C*. This guarantees safe neighbor traversal without allowing modification.
- `context_everywhere dist != null ** ... Perm(dist[i], write)` Provides write permission to the distance array. Full permission is required because BFS updates distances.

- `context_everywhere ... { :R[v]: } < C.length`; Ensures all adjacency offsets are valid indices.
- `context_everywhere ... { :C[v]: } < dist.length`; Guarantees that every edge refers to an existing vertex.

Preconditions

- `requires dist.length == n`; Ensures the distance array covers all vertices.
- `requires R.length == n + 1`; Establishes a valid compressed adjacency structure.
- `requires 0 <= s && s < n`; Guarantees the source vertex is within bounds.

These conditions eliminate the possibility of out-of-bounds memory access.

Initialization Loop

- `loop_invariant 0 <= i && i <= n`; Ensures safe iteration while initializing distances.
- The loop writes to `dist[i]`, which is allowed due to the global write permission.

After initialization, all vertices are marked as unvisited.

Source Initialization

- `dist[s] = 0`; This establishes the BFS starting level.

Main Loop

- `loop_invariant done == 0 || done == 1`; Restricts the control variable to Boolean values, simplifying reasoning about termination behavior.
- The loop repeatedly propagates distances until no updates occur.

Vertex Iteration

- `loop_invariant 0 <= v && v <= n`; Ensures safe traversal of all vertices.
- The condition `if (dist[v] == iteration)` selects vertices belonging to the current BFS frontier.

Adjacency Range Safety

- Loop invariants ensure:
 - `v + 1 < R.length` Safe access to `R[v+1]`.
 - `0 <= off <= C.length` Safe traversal of adjacency entries.

Thus, VerCors proves that the BFS implementation is free from out-of-bounds access and respects the specified permission discipline.

```

1 context_everywhere R != null ** (\forallall* int i; 0<=i && i<R.length; Perm({:R[i]:}, read
   ));
2 context_everywhere C != null ** (\forallall* int i; 0<=i && i<C.length; Perm({:C[i]:}, read
   ));
3 context_everywhere dist != null ** (\forallall* int i; 0<=i && i<dist.length; Perm({:dist[i]
   :}, write));
4 requires n >= 0;
5 requires dist.length == n;
6 requires R.length == n + 1;
7 requires C.length >= 0;
8 requires 0 <= s && s < n;
9 context_everywhere (\forallall int v; 0 <= v && v < R.length; 0 <= {:R[v]:} && {:R[v]:} < C
   .length);
10 context_everywhere (\forallall int v; 0 <= v && v < C.length; 0 <= {:C[v]:} && {:C[v]:} <
   dist.length);
11
12 void bfs(int n, int[] R, int[] C, int s, int[] dist)
13 {
14     loop_invariant 0 <= i && i <= n;
15     for (int i = 0; i < n; i = i + 1)
16     {
17         dist[i] = 1000000000;
18     }
19
20     dist[s] = 0;
21
22     int iteration = 0;
23     int done = 0;
24     loop_invariant done == 0 || done == 1;
25     while (done == 0) {
26         done = 1;
27
28         loop_invariant 0 <= v && v <= n;
29         loop_invariant done == 0 || done == 1;
30         for (int v = 0; v < n; v = v + 1) {
31             if (dist[v] == iteration) {
32                 done = 0;
33                 loop_invariant 0 <= v && v < n;
34                 loop_invariant v + 1 < R.length;
35                 loop_invariant 0 <= R[v] && R[v] < C.length;
36                 loop_invariant 0 <= R[v + 1] && R[v + 1] <= C.length;
37                 loop_invariant 0 <= off && off <= C.length;
38                 for (int off = R[v]; off < R[v + 1]; off = off + 1) {
39                     int j = C[off];
40                     if (dist[j] == 1000000000) {
41                         dist[j] = iteration + 1;
42                     }
43                 }
44             }
45         }
46
47         iteration = iteration + 1;
48     }
49 }

```

Listing 4.10: Verified BFS implementation in VerCors

Chapter 5

The implementation of the Hitchhiking algorithm in PVL

This chapter documents the PVL implementation of the Hitchhiking algorithm. We focus on the structure of the implementation and its memory safety specification: how the graph and the algorithm's state are represented, and how permissions are used to guarantee safe access to memory. This chapter only covers the basic implementation of the algorithm; how the implementation is proven correct is discussed in the following chapters: Chapter 6 on soundness, Chapter 7 on completeness, and Chapter 8 on termination. The gradual implementation of the algorithm can be found in the GitHub repository ¹, with the final version of the implementation given in the file `hitchhiking.pvl` ², and reproduced in full in Appendix B, Listing B.1.

5.1 Why PVL

VerCors supports several input languages, including Java, C, OpenCL, and its own language PVL. We use PVL because a long-term goal of this thesis is to verify the parallel, GPU-based version of the Hitchhiking algorithm. PVL supports parallel constructs such as `par` blocks, and its underlying permission-based separation logic allows VerCors to reason about data-race freedom and memory safety for concurrent programs, in addition to functional correctness [1].

5.2 The Graph Representation

The graph is represented using the Compressed Sparse Row (CSR) format [13]. Two sequences, R and C , together encode it: C stores the outgoing edges of every node, and R stores, for each node, the offset into C at which its edges begin.

Listing 5.1 shows `isEdge`, which states that there is an edge from u to v if v appears somewhere among the entries of C between offsets $R[u]$ and $R[u+1]$, i.e., in u 's slice of the edge array.

Listing 5.2 shows `degree`, which follows directly from the same slicing: the number of outgoing edges of u is the size of its slice in C .

¹<https://github.com/mariam606/Formal-Verification-of-the-Hitchhiking-Algorithm-Using-VerCors>

²<https://github.com/mariam606/Formal-Verification-of-the-Hitchhiking-Algorithm-Using-VerCors/blob/main/hitchhiking.pvl>

```

48 requires 0 <= u && u < |R| - 1;
49 requires 0 <= R[u] && R[u+1] <= |C|;
50 pure bool isEdge(seq<int> R, seq<int> C, int u, int v) =
51   (\exists int j; R[u] <= j && j < R[u+1]; {C[j]:} == v);

```

Listing 5.1: Checking whether an edge exists

```

53 // number of successors of node u in CSR
54 inline pure int degree(seq<int> R, int u) = R[u+1] - R[u];

```

Listing 5.2: Computing a node's degree

Listing 5.3 shows `validGraph`, which states the well-formedness conditions this CSR encoding must satisfy for N nodes: R has exactly one entry per node plus a final offset, the first offset is 0 and the last equals the size of C , offsets are non-decreasing and stay within bounds, and every entry of C is a valid node index. This predicate is assumed as a `context_everywhere` by every function that takes the graph as input.

CSR is a natural choice for a parallel algorithm. Since the edges of each node are stored contiguously, and the offset of every node is known in advance, a node's neighbours can be read directly, without following pointers or search structures. This makes it possible for different threads to process different nodes independently and access memory in a predictable pattern, which is why CSR is commonly used in parallel graph algorithms, for example in GPU implementations of parallel BFS [13].

In the current implementation, R and C are given the PVL type `seq<int>`, an immutable mathematical sequence. This is sufficient for now, since the graph is only ever read. For the parallel version of the algorithm, however, R and C will need to become `int[]` arrays instead. In VerCors, reasoning about concurrent access to memory is done through permissions (`Perm`) on heap locations, and a `par` block distributes these permissions among threads. A `seq<int>` is a pure, ghost-level value and cannot be the target of a permission or be split across threads. To later verify a `par` block in which multiple threads read the graph concurrently, R and C must be represented as real heap arrays with explicit permissions, matching how the graph would actually be stored in GPU global memory.


```

56 inline pure bool validGraph(int N, seq<int> R, seq<int> C) =
57   |R| == N + 1 &&
58   R[0] == 0 &&
59   |C| == R[N] &&
60   (\forall forall int u; 0 <= u && u < N; {R[u]:} <= R[u+1]) &&
61   (\forall forall int u; 0 <= u && u <= N; 0 <= {R[u]:} && R[u] <= |C|) &&
62   (\forall forall int i; 0 <= i && i < |C|; 0 <= {C[i]:} && C[i] < N);

```

Listing 5.3: Well-formedness of the CSR graph encoding

5.3 The Representation of the Algorithm

Besides the graph, the algorithm maintains its state in four arrays, corresponding directly to the notation used in Chapter 3:

- p represents the p -value array. Each entry $p[v]$ holds either a node index (a reference to the maximal accepting predecessor of the node), or one of two special values: $BOTTOM()$, meaning v has not been visited yet, and $EPSILON()$, meaning v has been visited but has not (yet) been assigned a node reference.
- in_open is a 0/1 flag array representing the open set $\mathcal{O}$: $in_open[v] == 1$ means v is currently queued for exploration.
- in_active is a 0/1 flag array representing the active set $\mathcal{A}$: $in_active[v] == 1$ means v is an accepting state for which cycle detection is ongoing.
- $in_interrupted$ is a 0/1 flag array representing the interrupted set $\mathcal{F}$: $in_interrupted[v] == 1$ means the search for v was interrupted during the current round.

5.4 Specification Predicates

Because several functions share the same arrays and inputs, well-formedness is stated once and reused via a set of helper predicates, rather than restated by hand in every function's contract.

```

77 inline pure bool arr(int[] a, int n) = a != null && a.length == n;

```

Listing 5.4: Array shape check

Listing 5.4 shows `arr`, which checks that an array is non-null and has the expected length n .

```

82 inline pure bool allEq(int[] a, int n, int val) =
83   (\forall forall int v; 0 <= v && v < n; {a[v]:} == val);

```

Listing 5.5: Checking that an array is uniformly initialised

Listing 5.5 shows `allEq`, which checks that every entry of an array equals a given value. It is used to state the initial contents of the state arrays, e.g., that p starts out equal to $BOTTOM()$ everywhere.

```

95 inline pure bool flags01(int[] a, int n) =
96   (\forall forall int v; 0 <= v && v < n; {a[v]:} == 0 || a[v] == 1);

```

Listing 5.6: Checking a 0/1 flag array

Listing 5.6 shows `flags01`, which checks that every entry of a 0/1 flag array is indeed 0 or 1, used for `in_open`, `in_active`, and `in_interrupted`.

```

85 inline pure bool validInput(int num_nodes, seq<int> is_red, seq<int> R, seq<int> C) =
86   num_nodes > 0 &&
87   |is_red| == num_nodes &&
88   validGraph(num_nodes, R, C) &&
89   (\forall int v; 0 <= v && v < num_nodes; {is_red[v]:} == 0 || is_red[v] == 1);
90
91 inline pure bool validRoots(int num_roots, seq<int> roots, int num_nodes) =
92   num_roots > 0 && |roots| == num_roots &&
93   (\forall int i; 0 <= i && i < num_roots; 0 <= {roots[i]:} && roots[i] < num_nodes);

```

Listing 5.8: Well-formedness of the algorithm's inputs

```

98 inline pure bool pWellFormed(int[] p, seq<int> is_red, int n) =
99   (\forall int v; 0 <= v && v < n;
100    {p[v]:} == BOTTOM() || p[v] == EPSILON() || (0 <= p[v] && p[v] < n)) &&
101   (\forall int v; 0 <= v && v < n;
102    (0 <= {p[v]:} && p[v] < n) ==> is_red[p[v]] == 1);

```

Listing 5.10: Well-formedness of the p-value array

```

79 inline pure bool validAlgoArrays(int[] p, int[] in_open, int[] in_active, int[]
80   in_interrupted, int n) =
   arr(p, n) && arr(in_open, n) && arr(in_active, n) && arr(in_interrupted, n);

```

Listing 5.7: Well-formedness of the algorithm's state arrays

Listing 5.7 shows `validAlgoArrays`, which bundles `arr` for all four state arrays at once.

Listing 5.8 shows `validInput`, which states that the graph is non-empty, well-formed, and that `is_red` is a proper 0/1 array of the right length, and `validRoots`, which states that the given root nodes are a non-empty, in-bounds subset of the graph's nodes. Both are assumed as `context_everywhere` wherever the corresponding inputs appear.

Beyond these array-shape and input predicates, further predicates state consistency between the state arrays themselves.

```

107 inline pure bool queueOk(seq<int> q, int[] p, int n) =
108   (\forall int j; 0 <= j && j < |q|; 0 <= {q[j]:} && q[j] < n) &&
109   (\forall int j; 0 <= j && j < |q|; {p[q[j]]:} != BOTTOM());

```

Listing 5.9: Well-formedness of the open queue

Listing 5.9 shows `queueOk`, which states that every node in the open queue `q` is a valid node index, and has already been assigned a p-value (i.e., is not `BOTTOM()`).

Listing 5.10 shows `pWellFormed`, which states that every entry of `p` is either one of the two special values or a valid node index, and that whenever it is a node index, that node is an accepting (red) state.

```

104 inline pure bool activeImpliesRed(int[] in_active, seq<int> is_red, int n) =
105   (\forall int v; 0 <= v && v < n; {in_active[v]:} == 1 ==> is_red[v] == 1);

```

Listing 5.11: Active states are accepting states

Listing 5.11 shows `activeImpliesRed`, which states that every node marked active is an accepting (red) state.

Combining the predicates: `stateInv`

Restating all of the predicates above by hand in the contract of every function would be repetitive and error-prone, since a function that forgets one of them would silently be allowed to violate it. Instead, the core predicates are bundled into a single invariant, `stateInv` in 5.12

```

127 inline pure bool stateInv(int n, seq<int> R, seq<int> C, seq<int> is_red, seq<int> q,
128                          int[] p, int[] in_open, int[] in_active, seq<int>[] ghost_path
129                          , int m) =
129     flags01(in_open, n) &&
130     flags01(in_active, n) &&
131     queueOk(q, p, n) &&
132     pWellFormed(p, is_red, n) &&
133     activeImpliesRed(in_active, is_red, n) &&
134     ghostPathsOk(n, R, C, ghost_path, p) &&
135     maxActiveOk(p, in_active, m, n);

```

Listing 5.12: The state invariant

The first five conjuncts of `stateInv` are exactly the structural predicates just introduced: they state that the algorithm's arrays have the right shape and are internally consistent with each other (e.g., an active node is always red, a queued node always has a real p-value).

`stateInv` is included in the `context_everywhere` clause, directly or via the composite predicates described next, of `processRoots`, `processNode`, `resetActive`, and `hitchhiking`, and is proven to be preserved by every loop in these functions. This gives two guarantees for the rest of the implementation.

First, memory safety: since every array access is guarded by `arr` (via `validAlgoArrays`) and the index bounds implied by `pWellFormed` and `queueOk`, VerCors can guarantee no out-of-bounds access or null dereference occurs anywhere in the algorithm. Second, a stable foundation for correctness: the soundness proof in Chapter 6 and the completeness proof in Chapter 7 both rely on `stateInv` already holding at every point they reason about.

5.5 Function-by-function walkthrough

The algorithm is split into functions, mirroring the structure of Algorithm 2 in [14], reproduced in Appendix A, Algorithm 1.

processRoots. `processRoots` implements the initialisation of Algorithm 2, lines 5–8: for every initial state, it is added to the open set $\mathcal{O}$ (line 6); if it is an accepting state, it is added to the active set $\mathcal{A}$ and its p-value is set to itself (line 7), otherwise its p-value is set to `EPSILON()` (line 8). It returns the resulting open queue.

processNode. `processNode` implements the body of the round, Algorithm 2, lines 10–21, for a single node taken from the open set. For each child of that node, it checks whether an accepting cycle has been found (line 13); if so, it sets a flag, yields the detected cycle as a ghost witness, and returns immediately. Otherwise, it handles a first visit to an accepting child by marking it active (lines 14–15), determines whether the current search continues into the child or is interrupted (lines 16–17), atomically updates the child's p-value to the maximum of the current and new value (line 18), records further interruptions where required (lines 19–20), and re-adds the child to the open queue if its p-value increased (line 21).

anyInterrupted. `anyInterrupted` corresponds to the condition “if $\mathcal{F} \neq \emptyset$ ” at the start of the post-processing step, Algorithm 2, line 22. Since $\mathcal{F}$ is represented by the flag array `in_interrupted` rather than as an explicit set, this function scans the array to determine whether the condition holds.

resetActive. `resetActive` implements the post-processing of a round, Algorithm 2, lines 23–28: for every node, it decides whether an active, interrupted node must be reopened and re-added to the open set (lines 24–25), removes nodes from $\mathcal{A}$ that are not reopened (line 26), clears the interrupted flag (line 27), and resets the p-value of every node depending on whether it remains active (line 28).

hitchhiking. The main function implements the outer structure of Algorithm 2. It calls `processRoots` for the initialisation of lines 5–8, then repeatedly processes the open queue (the round of lines 9–21) by calling `processNode` on each node and returning immediately if a cycle is found (line 13). Once the open queue is empty, it calls `anyInterrupted` and, if necessary, `resetActive`, implementing the post-processing of lines 22–28. If a round completes with an empty open queue and no interruptions, the function returns, reporting that no accepting cycle exists (line 29).

Chapter 6

Verification of Soundness

This chapter presents the formal verification of the soundness property of the sequential Hitchhiking algorithm. Soundness corresponds to the $\Rightarrow$ direction established in the proof of Theorem 1 in [14]:

Theorem 6.1 (Soundness). *If hitchhiking returns true, then there exists an accepting cycle in the graph.*

In plain terms, soundness says the algorithm never lies: every time it reports a counterexample, one genuinely exists.

6.1 Overview of how to prove

Across the entire implementation, there is only one place where a cycle is detected and reported: a single line inside `processNode`, reached while a node is processing one of its children, which checks whether the p-value assigned to the node equals that same child. Proving soundness therefore comes down to proving that, at this one line, if the check succeeds, an accepting cycle really exists.

We use the graph in Figure 6.1 as a running example throughout this chapter. It has five nodes, 0 to 4, and edges $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, and $4 \rightarrow 0$. Node 1 is red, i.e., an accepting state; the other nodes are not. This graph contains exactly one cycle, $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$, and since it passes through the red node 1, it is an accepting cycle.

Suppose Hitchhiking starts exploring from node 0. As the algorithm propagates the p-value 1 (the only accepting node) forward along the cycle, the p-values of nodes 2, 3, and 4 are all eventually set to 1, and finally the p-value of node 0 is set to 1 as well, once the propagation reaches all the way back around to node 0. At that point, node 0 is reprocessed: its only child is node 1, and its p-value is now 1, the same as that child. This is exactly the situation in which Hitchhiking detects an accepting cycle: a node whose p-value equals one of its own children.

For this detection to genuinely mean an accepting cycle exists, three facts must hold simultaneously, illustrated here on the example before any code is shown:

1. **The shared value is a red node.** In the example, the shared value is 1, and node 1 is red.
2. **There is a path from that shared value to the node being processed.** In the example, there is a path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0$ from 1 to 0.
3. **There is an edge from the node being processed back to the shared value.** In the example, there is an edge $0 \rightarrow 1$.

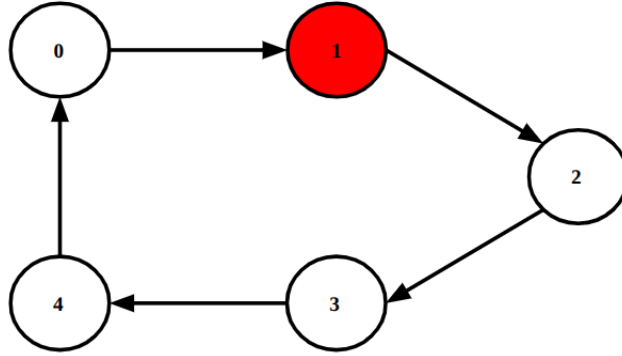


Figure 6.1: An example graph containing one accepting cycle.

Together, these three facts describe exactly the cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow 1$: a path from a red node back to itself. When hitchhiking detects this situation, it does not merely report `true`; it also returns the cycle itself, the path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow 1$ in the example, as a concrete witness.

6.2 Proof sketch

The proof that these three facts hold rests on two things the implementation tracks alongside the `p`-value array as it runs:

- **The array `p` is well-formed.** Every entry of `p` is either a special value (epsilon or bottom), or a real node index pointing to an accepting (red) node. This gives the first fact: if the shared value at the point of detection is a real node index, it is guaranteed to be red.
- **Every `p`-value is backed by a concrete path, recorded in a ghost array `ghost_path`.** Knowing that `p[v]` is a node index tells us *that* some accepting predecessor was found for `v`, but not *how* it was reached. To also capture this, the implementation maintains, alongside `p`, a ghost array `ghost_path`, where `ghost_path[v]` represents a concrete path in the graph from `p[v]` to `v`, updated every time `p[v]` is updated so that it always stays a faithful witness of how that `p`-value came to be. This gives the second fact: the path the algorithm needs is already sitting in `ghost_path`.
- **If we find a cycle, the path we kept track of is that cycle.** When the detection line is reached, `ghost_path` for the node being processed is already a path from the shared value back to that node. Combined with the edge from the node back to the shared value, the third fact, this path closes into a cycle. The algorithm yields this path as the concrete witness proving Theorem 6.1.

The following three sections prove each of these facts in turn, referring to the corresponding code and specifications, and the section after that shows how they are combined into the full soundness proof.

6.3 Proof of Condition 1: the shared value is red

Condition. Whenever the `p`-value `p[node]` equals a child of node, that child is a red node.

```

98 inline pure bool pWellFormed(int[] p, seq<int> is_red, int n) =
99   (\forallall int v; 0 <= v && v < n;
100    { :p[v]:} == BOTTOM() || p[v] == EPSILON() || (0 <= p[v] && p[v] < n)) &&
101   (\forallall int v; 0 <= v && v < n;
102    (0 <= { :p[v]:} && p[v] < n) ==> is_red[p[v]] == 1);

```

Listing 6.1: Well-formedness of the function p

```

1 int alpha = p[node];
2 int child = C[R[node] + ci];
3
4 if (alpha == child) {
5   found[0] = 1;
6   assert is_red[child] == 1;
7   ...
8 }

```

Listing 6.2: Establishing Condition 1

This condition follows from the invariant `pWellFormed` in Listing 6.1, which is maintained throughout `processRoots`, `processNode`, and `resetActive` as part of `stateInv`, introduced in Chapter 5

As explained, `BOTTOM()` and `EPSILON()` are two special values used before a node has been assigned a real p-value: `BOTTOM()` means the node has not been reached at all, and `EPSILON()` means it has been reached but not yet assigned an accepting predecessor. The first line of `pWellFormed` states that every entry of `p` is one of these two special values, or a valid node index. The second line states that whenever it is a valid node index, that node is red.

In the code, the check happens inside `processNode`, at the point where a node's p-value is compared to one of its children, as shown in Listing 6.2

Here, `alpha` is `p[node]` and `child` is the current child being processed. Since `alpha == child`, `child` is a real node index (not a special value), so `pWellFormed` directly guarantees `is_red[alpha] == 1`, i.e., `is_red[child] == 1`. This is exactly Condition 1, confirmed by the `assert`. In the example, this is precisely the step of concluding `is_red[1] == 1` from `p[0] == 1`.

6.4 Proof of Condition 2: a path from the shared value to the node

Condition. Whenever the p-value `p[node]` equals a child of node, there is a path in the graph from that child to node.

`pWellFormed` says nothing about paths: it only relates a p-value to the fact that the node it points to is red, not to how that value was reached. To also capture the path, the implementation maintains a ghost array `ghost_path` alongside `p`. `ghost_path` is a ghost variable: it exists only at the specification level, to record the actual path each time a p-value is set, and has no effect on the algorithm's runtime behaviour. In the example, once `p[0]` is set to 1, `ghost_path[0]` is set to the sequence `[1,2,3,4,0]`, the concrete path that was propagated. This is guaranteed in general by the invariant `ghostPathsOk` in Listing 6.3, also maintained as part of `stateInv`

Applied to $v = \text{node}$, `ghostPathsOk` states that whenever `p[node]` is a valid node index, `ghost_path[node]` is non-empty, starts at `p[node]`, ends at `node`, stays within bounds, and every consecutive pair of entries is connected by a real edge. In the code, this witness

```

111 inline pure bool ghostPathsOk(int n, seq<int> R, seq<int> C, seq<int>[] ghost_path, int
    [] p) =
112     (\forallall int v; 0 <= v && v < n;
113      (0 <= {p[v]:} && p[v] < n) ==>
114      |ghost_path[v]| >= 1 &&
115      ghost_path[v][0] == p[v] &&
116      ghost_path[v][|ghost_path[v]| - 1] == v &&
117      (\forallall int i; 0 <= i && i < |ghost_path[v]|;
118       0 <= {ghost_path[v][i]:} && ghost_path[v][i] < n) &&
119      (\forallall int i; 0 <= i && i < |ghost_path[v]| - 1;
120       isEdge(R, C, {ghost_path[v][i]:}, ghost_path[v][i+1])));

```

Listing 6.3: Well-formedness of the ghost-path witnesses

```

1 if (alpha == child) {
2   found[0] = 1;
3   assert is_red[child] == 1;
4
5   ghost cycle_out = ghost_path[node];
6   assert cycle_out[0] == child;
7   assert cycle_out[|cycle_out| - 1] == node;
8   assert (\forallall int i; 0 <= i && i < |cycle_out| - 1;
9           isEdge(R, C, {cycle_out[i]:}, cycle_out[i+1]));
10  ...
11 }

```

Listing 6.4: Establishing Condition 2

is bound to a ghost variable `cycle_out` and checked right after Condition 1 as shown in Listing 6.4

Since `p[node] == alpha == child`, `ghostPathsOk` guarantees that `cycle_out`, i.e., `ghost_path[node]`, is a valid path in the graph from `child` to `node`: it starts at `child`, ends at `node`, and every step follows a real edge. This is exactly Condition 2. In the example, this is precisely `ghost_path[0]` being the path `[1, 2, 3, 4, 0]`.

6.5 Proof of Condition 3: an edge back to the shared value

Condition. There is an edge in the graph from `node` to `child`.

Unlike Conditions 1 and 2, this condition does not depend on any invariant maintained across the algorithm's execution. It follows directly from how `child` was obtained in the first place, as shown in Listing 6.5

```

391 int child = C[R[node] + ci];
392 assert isEdge(R, C, node, child);

```

Listing 6.5: Establishing Condition 3

Since `child` is read as `C[R[node] + ci]`, it is a neighbour of `node` by construction of the CSR encoding: this is exactly what the `assert isEdge(R, C, node, child)` states, verified using the `isEdge` and `validGraph` predicates from Chapter 2. In the example, this is the fact that 1 is indeed a neighbour of 0, i.e., the edge `0 → 1` exists.

6.6 Wrapping up: the returned witness

With all three conditions established, `cycle_out` satisfies `isAcceptingCycle`: Condition 2 gives a valid path from `child` to `node`; Condition 3 gives the closing edge from `node` back to

child; and Condition 1 gives a red node on that path, namely child itself. In the example, this is exactly the cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow 1$ from Figure 6.1. This is captured by the postcondition of `processNode`, as shown in Listing 6.6

```
365 ensures found[0] == 1 ==> isAcceptingCycle(num_nodes, R, C, is_red, cycle_out);
```

Listing 6.6: Soundness postcondition of `processNode`

This postcondition is propagated to `hitchhiking` through the `given/yields` clause on the call to `processNode`, binding `cycle_out` to `hitchhiking`'s own ghost variable `cycle`. Whenever `found[0] == 1`, `hitchhiking` returns `true` with `cycle` satisfying `isAcceptingCycle`, as shown in Listing 6.7

```
472 yields seq<int> cycle;  
473 ensures \result ==> isAcceptingCycle(num_nodes, R, C, is_red, cycle);
```

Listing 6.7: Soundness postcondition of `hitchhiking`

This is precisely the statement of Theorem 6.1: not only does `hitchhiking` report `true` only when an accepting cycle exists, it hands back that cycle itself as a checkable witness. Every annotation in this proof is verified automatically by VerCors; no `assume` statements are used anywhere, and no part of the proof is left as an unverified assumption. This answers the first research question (RQ1): how VerCors can be used to verify the soundness of the sequential Hitchhiking algorithm.

Chapter 7

Verification of Completeness

This chapter presents the formal verification of the completeness property of the sequential Hitchhiking algorithm. Completeness is the other direction of Theorem 1 in [14]:

Theorem 7.1 (Completeness). *If there exists an accepting cycle in the graph, then hitchhiking returns true.*

Equivalently, by contraposition: if hitchhiking returns false, then there does not exist an accepting cycle in the graph. Where soundness (Chapter 6) says the algorithm never reports a cycle that does not exist, completeness says the reverse: the algorithm never misses a cycle that does exist.

[14] proves completeness by building up six lemmas (Lemmas 1 to 6 in the paper). Lemma 1 is a termination result for a single round of the algorithm, needed to make the algorithm's overall termination possible in the first place (partial correctness, i.e., soundness and completeness together, only says what happens *if* hitchhiking terminates; Lemma 1 is a main ingredient of the separate argument for termination itself, which combines with partial correctness into total correctness). Its verification is therefore left for the termination chapter 8, rather than presented here.

This chapter covers the remaining five lemmas, Lemmas 2 to 6, each establishing one property about how the algorithm's state evolves over a run. Lemmas 2, 3, and 5 are fully verified in PVL, with no assume statements anywhere in their proofs. Lemma 4 is left for future work, for the reasons discussed in Section 7.4, and Lemma 6, which depends directly on Lemma 4, is left for future work as a consequence.

7.1 Overview: what the lemmas prove and how they combine

This section gives a high-level picture of what each lemma is for and how, together, they prove completeness, following the argument given for Theorem 1 in [14].

The algorithm maintains a set $\mathcal{A}$ of accepting states for which a cycle-detection search is currently underway, and reasons about $\bar{q}_{\max}$, the largest element of $\mathcal{A}$ according to the fixed order on red nodes. This is a device used only in the proof; the algorithm itself never computes $\bar{q}_{\max}$ explicitly.

- **Lemma 3** makes $\bar{q}_{\max}$ meaningful as “the current top candidate”: as long as $\mathcal{A}$ is non-empty, no p-value and no active node can ever exceed $\bar{q}_{\max}$. Every other lemma relies on this bound to reason about what $\bar{q}_{\max}$'s search can and cannot be interrupted by.

- **Lemma 2** guarantees that, from the end of the first round onward, every p-value being compared in this way is a real value, not the placeholder for a node that has not been visited yet, so these comparisons are meaningful.
- **Lemma 4** says that if $\bar{q}_{\max}$ itself lies on an accepting cycle, the round cannot finish without detecting that cycle: because nothing can outrank $\bar{q}_{\max}$ (Lemma 3), its search is never blocked, and it keeps expanding until it reaches every node reachable from it, including, if it is on a cycle, the edge that closes back onto itself.
- **Lemma 5** says that, for the same reason, $\bar{q}_{\max}$ itself is never interrupted: it can never be on the losing side of the comparisons that populate the interrupted set $\mathcal{F}$. Consequently, whenever a round ends without finding a cycle, $\bar{q}_{\max}$ is unconditionally dropped from $\mathcal{A}$ during post-processing.
- **Lemma 6** guarantees that this dropping never costs the algorithm an undetected cycle: for every accepting cycle not yet found, at least one of its own active nodes is interrupted and ready to be reopened at the end of every round, so a fresh candidate belonging to that cycle survives into the next round.

Combining these: every round that ends without a counter-example being found drops the current $\bar{q}_{\max}$ from $\mathcal{A}$ (Lemma 5), and by Lemma 4 this can only happen when $\bar{q}_{\max}$ was *not* on any accepting cycle. So each such round removes one candidate that is provably not part of the problem. At the same time, Lemma 6 guarantees that no accepting cycle ever loses its last active representative, so this removal can never eliminate a cycle from consideration. Since $\mathcal{A}$ is finite and keeps losing exactly the candidates that are not on any cycle, this cannot continue forever: eventually every node left in $\mathcal{A}$ must lie on some accepting cycle, so the current $\bar{q}_{\max}$ does too, and Lemma 4 then forces hitchhiking to return true. If no accepting cycle exists in the first place, this elimination process simply continues until $\mathcal{A}$ empties out entirely, and the algorithm correctly reports no counter-example.

7.2 Lemma 2: every node eventually has a real p-value

The second lemma needed for completeness is stated in [14] as follows:

When l.22 is reached for the first time, $\forall \bar{q} \in Q_{\otimes}. p(\bar{q}) \neq \perp$ holds until Hitchhiking terminates.

In plain terms: by the time the first round finishes without returning, no node's p-value is still BOTTOM(), and this stays true for the rest of the run. Several later lemmas compare p-values against each other; those comparisons only make sense once every node being compared has actually been assigned a value.

Why one round already reaches every node. The key observation in [14]'s proof is that a single round already visits the entire part of the graph reachable from the roots. A round does not process the states present in the open set just once and stop: it keeps taking states out and processing them, including states that are discovered and added back to the open set *during* that same round, until the open set is genuinely empty. Every time a state is processed, every one of its successors has its p-value touched, via the atomic maximum update at the point corresponding to line 18 of the pseudocode; any successor visited for the first time is, in the same step, added back to the open set to be processed in its own turn. So the first round does not stop at some fixed exploration depth: it keeps expanding until nothing new is left to discover, at which point every reachable node has already had its p-value moved away from BOTTOM().

Persistence: once real, always real. The second half of the lemma, that a real value is never lost again, follows from how the maximum-update at line 18 is defined: `BOTTOM()` is the bottom element of the order on p-values, and every update takes a maximum, so a value can only move away from `BOTTOM()`, never back into it. In the PVL implementation, this is exactly what the postcondition of `processNode` states, as shown in Listing 7.1.

```
366 ensures (\forall int w; 0 <= w && w < num_nodes; \old(p[w]) != BOTTOM() ==> { :p[w]: } !=
      BOTTOM());
```

Listing 7.1: `processNode` never re-introduces `BOTTOM()`

Since `processNode` is the only function that changes `p` while a round is in progress, and neither `processRoots` nor `resetActive` ever writes `BOTTOM()` into an entry that already holds a real value, chaining this postcondition across every call to `processNode` in every round is what keeps the property alive for the rest of the run.

7.3 Lemma 3: the maximum active node bounds every p-value

The third lemma needed for completeness is stated in [14] as follows:

$$\mathcal{A} \neq \emptyset \implies \forall \bar{q} \in Q_{\otimes}. \bar{q}_{\max} \geq p(\bar{q}) \text{ is an invariant of Hitchhiking.}$$

In plain terms, as long as some node is active, $\bar{q}_{\max}$ is always at least as large as every p-value currently assigned anywhere in the graph. This is the bound every other lemma leans on: Lemma 4 and Lemma 5 both work by showing that $\bar{q}_{\max}$'s search can never lose a comparison, and Lemma 3 is precisely what makes that true.

Since $\mathcal{A}$ and $\bar{q}_{\max}$ are reasoning devices from the paper's proof, not values the algorithm tracks at runtime, the PVL implementation introduces a ghost variable to stand in for $\bar{q}_{\max}$, shown in Listing 7.2.

```
488 ghost int max_active = BOTTOM();
```

Listing 7.2: Ghost variable tracking the largest node ever activated

`max_active` starts at `BOTTOM()`, matching the fact that no node has been activated yet. It is only ever increased, and only at the two places where a node is actually added to the active set: while seeding roots, and while processing a node's children, as shown in Listings 7.3 and 7.4.

```
229 ghost if (r > max_active_out) { max_active_out = r; }
```

Listing 7.3: Raising `max_active` while seeding roots

```
414 ghost if (child > max_active_out) { max_active_out = child; }
```

Listing 7.4: Raising `max_active` while processing a child

In `processNode`, the update in Listing 7.4 happens before the check that decides whether `alpha` should be updated to `child`, so that by the time `alpha` is set to `child`, `max_active` has already been raised to account for it if needed.

Lemma 3 itself, together with the supporting facts needed to maintain it, is expressed by the predicate `maxActiveOk`, one of the conjuncts of `stateInv` from Chapter 5, and carried through every loop in the implementation as part of it, as shown in Listing 7.5.

The first conjunct states that `max_active` (bound to `m` here) is always either `BOTTOM()` or a valid node index. The second conjunct is Lemma 3 itself: whenever `p[w]` is a real node index rather than one of the two special values, it is at most `max_active`; writing

```

122 inline pure bool maxActiveOk(int[] p, int[] in_active, int m, int n) =
123   (m == BOTTOM() || (0 <= m && m < n)) &&
124   (\forall int w; 0 <= w && w < n; {p[w]:} >= 0 ==> p[w] <= m) &&
125   (\forall int v; 0 <= v && v < n; {in_active[v]:} == 1 ==> v <= m);

```

Listing 7.5: Bounding p-values and active nodes by the largest node ever activated

it as an implication, rather than asserting the bound directly, is what correctly excludes `BOTTOM()` and `EPSILON()`, which are never meant to be compared against `max_active` as node indices. The third conjunct states that every currently active node is at most `max_active`, needed specifically to justify `resetActive`: when it sets `p[v] = v` for an active node, re-establishing the second conjunct for that node requires already knowing $v \leq \text{max_active}$, which this third conjunct supplies.

`max_active` and `maxActiveOk` are threaded through `processRoots`, `processNode`, and `resetActive` using VerCors' given/yields mechanism, so the value computed inside one function call is passed on to the next. Since `max_active` only ever increases, and every place it changes is matched by a corresponding invariant update, Lemma 3 holds continuously across every round of the algorithm's execution.

7.4 Lemma 4: the round in which the maximum active node lies in a cycle

The fourth lemma needed for completeness is stated in [14] as follows:

At l.9: If $\bar{q}_{\max} \in \mathcal{C}$ and $\forall \bar{q} \in Q_{\otimes}. p(\bar{q}) \neq \perp$, then Hitchhiking returns at l.13 before l.22 is reached.

Here $\mathcal{C}$ refers to the set of all nodes that lie on some accepting cycle. Lemma 4 says that if $\bar{q}_{\max}$ lies on an accepting cycle at the start of a round, and every node has already been visited, then that round is guaranteed to find the cycle before finishing. This follows from Lemma 3. No other p-value can exceed $\bar{q}_{\max}$, so $\bar{q}_{\max}$'s search is never blocked and keeps expanding until it traverses the edge that closes the cycle back onto itself.

7.4.1 Status: left for future work

Proving Lemma 4 in PVL is left for future work due to time constraints. My suggestion for a target formalisation is Listing 7.6.

Listing 7.6 states that no accepting cycle contains `qmax`. It reuses `isAPath` and `isEdge`, the same building blocks `isAcceptingCycle` uses in Chapter 2. The target is an assertion stating that `max_active_end_round`, introduced in Section 7.5, satisfies `noAcceptingCycle` at the end of every round that does not return. This is the contrapositive of Lemma 4.

```

1 requires validGraph(N, R, C);
2 requires |is_red| == N;
3 inline pure bool noAcceptingCycle(int N, seq<int> R, seq<int> C, seq<int> is_red, int
   qmax) =
4   (\forall forall seq<int> cycle; isAPath(N, R, C, cycle) && isEdge(R, C, cycle[|cycle| - 1],
   cycle[0])
5   && (\exists int i; 0 <= i && i < |cycle|; is_red[{:cycle[i]:}] == 1);
6   !(qmax in cycle));

```

Listing 7.6: A target formalisation of Lemma 4, for future work

7.5 Lemma 5: the maximum active node is never interrupted

The fifth lemma needed for completeness is stated in [14] as follows:

At l.22: $\bar{q}_{\max} \notin \mathcal{C} \wedge \bar{q}_{\max} \notin \mathcal{F}$.

This lemma has two conjuncts. The first conjunct, $\bar{q}_{\max} \notin \mathcal{C}$, follows from the contrapositive of Lemma 4 together with Lemma 2, and is not independently verified in PVL. The second conjunct, $\bar{q}_{\max} \notin \mathcal{F}$, is proven here. The interrupted set $\mathcal{F}$ is only ever populated by the losing side of a comparison between two competing values, and Lemma 3 guarantees $\bar{q}_{\max}$ can never be the losing side. So $\bar{q}_{\max}$ can never end up in $\mathcal{F}$.

To state a per-round version of this lemma, the implementation introduces a ghost variable, `max_active_end_round`, capturing the value of `max_active` once every node in the open queue has been processed and no cycle has been found. Listing 7.7 initialises it before the round loop, and Listing 7.8 captures it right before post-processing begins, corresponding to line 22 of the pseudocode.

```

499 ghost int max_active_end_round = num_nodes;

```

Listing 7.7: `max_active_end_round` initialised before the round loop

```

539 max_active_end_round = max_active;

```

Listing 7.8: `max_active_end_round` captured at the end of a round

Listing 7.9 gives the assertion proven in PVL. It states that `max_active_end_round` is either unset or a valid node index whose interrupted flag is 0. It is placed right after the child-processing loop of a round finishes, at the same point where `max_active_end_round` is set above.

```

542 assert max_active_end_round == BOTTOM() || (0 <= max_active_end_round &&
   max_active_end_round < num_nodes && in_interrupted[max_active_end_round] ==
   0);

```

Listing 7.9: Lemma 5 as a direct assertion

This assertion follows from `interruptionInv`, shown in Listing 7.10, one of the conjuncts of `processNodeInv` carried through the child-processing loop inside `processNode`.

Instantiated with `m := max_active`, the last conjunct of Listing 7.10 is exactly the fact needed. Maintaining it rules out `max_active` being added to `in_interrupted` at each of the three sites where this can happen, using `maxActiveOk` from Section 7.3.

- Listing 7.11 interrupts a child only on its first visit, when its p-value is still `BOTTOM()`. `max_active`'s own p-value already equals itself and is never `BOTTOM()`, so it cannot be the child interrupted here.

```

137 requires m == BOTTOM() || (0 <= m && m < n);
138 inline pure bool interruptionInv(int n, int[] p, int[] in_active, int[] in_interrupted,
    int m) =
139   flags01(in_interrupted, n) &&
140   (\forall forall int i; 0 <= i && i < n; {:in_interrupted[i]:} == 1 ==> in_active[i] == 1)
    &&
141   (\forall forall int w; 0 <= w && w < n; 0 <= {:p[w]:} && p[w] < n ==> in_active[p[w]] == 1)
    &&
142   (m == BOTTOM() ==> (\forall forall int i; 0 <= i && i < n; {:in_interrupted[i]:} == 0)) &&
143   (m == BOTTOM() || in_interrupted[m] == 0);

```

Listing 7.10: interruptionInv: the predicate supplying Lemma 5

```

411 // lines 14–17: first visit to a red child
412 if (p[child] == BOTTOM() && is_red[child] == 1) {
413   in_active[child] = 1;
414   ghost if (child > max_active_out) { max_active_out = child; }
415   if (child > alpha) {
416     alpha = child;
417   } else {
418     in_interrupted[child] = 1;
419   }
420 }

```

Listing 7.11: First interruption site: a first visit to a red child

- Listing 7.12 interrupts the previous value beta when alpha > beta. If beta were max_active, alpha would need to exceed it, which maxActiveOk forbids.
- Listing 7.13 interrupts alpha when beta > alpha, ruled out symmetrically, since beta is likewise always at most max_active.

```

433 // line 19: 2nd interruption
434 if (alpha > beta && beta > EPSILON() && in_open[child] == 1) {
435   in_interrupted[beta] = 1;
436 }

```

Listing 7.12: Second interruption site

```

438 // line 20: 3rd interruption
439 if (beta > alpha && alpha > EPSILON() && in_active[child] == 0) {
440   in_interrupted[alpha] = 1;
441 }

```

Listing 7.13: Third interruption site

The remaining case is when the child itself is about to become the new maximum. Listing 7.4, given in Section 7.3, raises max_active before the interruption check runs, so this child is always guaranteed to exceed the old value of alpha, and always continues the search instead of being interrupted.

With this invariant maintained throughout processNode, the assertion in Listing 7.9 holds without needing any assume, and it directly supplies the precondition resetActive needs on max_active.

7.6 Lemma 6: every accepting cycle eventually reopens one of its nodes

The sixth and final lemma needed for completeness is stated in [14] as follows:

At *I.22*: $\forall \sigma \in \Theta. \exists \bar{q} \in \mathcal{A} \cap \sigma. \bar{q} \in \mathcal{F} \wedge p(\bar{q}) \neq \bar{q}$.

Here Θ refers to the set of all accepting cycles in the graph. Lemma 6 says that for every accepting cycle, by the time post-processing is reached, at least one active node on that cycle has been interrupted and does not yet reference itself in p . This guarantees a fresh candidate on every undetected cycle survives into the next round.

Lemma 6 depends directly on Lemma 4. Its proof reasons about how a search along the cycle can only be interrupted by another search that itself traverses the same cycle, which requires knowing how the search rooted at $\bar{q}_{\max}$ behaves, exactly what Lemma 4 establishes. Verifying Lemma 6 is future work building upon Lemma 4: once Lemma 4 is verified in PVL, Lemma 6 can be proven on top of it.

7.7 Status of the Completeness Proof

This chapter presented the verification effort towards completeness of the sequential Hitchhiking algorithm, covering Lemmas 2 through 6 of the six-lemma structure in [14] (Lemma 1, on round termination, is covered separately in the termination chapter 8). Lemmas 2, 3, and 5 are fully verified in PVL, with no assume statements remaining anywhere in their proofs. Lemma 4 was left for future work, for the reasons discussed in Section 7.4, and Lemma 6, which depends directly on it, was left for future work as a consequence. Together, Lemmas 2, 3, and 5 already establish meaningful structural guarantees: every node is eventually assigned a real p -value, no p -value or active node can ever exceed the largest node ever activated, and that largest active node is never interrupted. What remains, Lemmas 4 and 6, is the part of the proof that ties these structural guarantees to the actual detection of every accepting cycle, by turning the domination argument behind Lemma 5 into a full reachability argument, and is the natural next step for continuing this verification effort.

Chapter 8

Verification of Termination

This chapter proves that hitchhiking terminates, completing the total correctness argument together with the partial correctness results of Chapters 6 and 7: partial correctness only says what happens if hitchhiking terminates, and this chapter is what makes that assumption hold. Section 8.1 proves Lemma 1 from [14], that every single round of the algorithm terminates. This says nothing about how many rounds the algorithm as a whole can take before it terminates entirely, nor about the small bounded loops used inside individual functions, so Section 8.2 covers the remaining simple loops, which need little justification, and Section 8.3 covers the loop that bounds the number of rounds itself, which does.

8.1 Lemma 1: termination of each round

In [14], this is stated as follows:

Every time the loop of l.10 is entered, and Hitchhiking does not return at l.13, eventually, l.22 is reached with $\mathcal{O} = \emptyset$.

In the PVL implementation, the loop of l.10 corresponds to the inner while loop inside `hitchhiking`, nested within the outer loop covered in Section 8.3. It drains `open_queue` for the current round, including any node rediscovered and added back to the queue while another node is being processed, until the queue is genuinely empty.

Intuition. Whenever a node is added back to the open queue, it is because processing some other node made an entry of `p` strictly grow, for instance from `BOTTOM()` to a real node index, or from one node index to a larger one. Every entry of `p` is bounded above by `num_nodes - 1`, and once increased, never decreases again. So the total amount by which every entry could ever grow, added up over the whole array, is itself bounded. Every re-addition to the open queue is paid for by some part of this fixed budget, which can only be spent, never replenished, so only finitely many re-additions can ever happen. Combined with the fact that the queue also shrinks whenever nothing is re-added, the round cannot run forever.

Expressing this in VerCors. This intuition is made precise as a lexicographic measure, shown in Listing 8.1.

522 **decreases** num_nodes * (num_nodes + 1) - sum_p, |open_queue| - ptr;

Listing 8.1: Termination measure for a round

```

7 requires 0 <= k && k <= |xs|;
8 requires (\forall forall int i; 0 <= i && i < |xs|; BOTTOM() <= {xs[i]:} && xs[i] < n);
9 ensures 0 <= \result;
10 ensures \result <= k * (n - BOTTOM() - 1);
11 decreases k;
12 pure int prefixSum(seq<int> xs, int k, int n) =
13   k == 0 ? 0 : prefixSum(xs, k - 1, n) + (xs[k - 1] - BOTTOM());

```

Listing 8.2: The recursive definition of the running sum

```

145 inline pure bool pSeqInv(int n, int[] p, seq<int> p_seq, int sum_p) =
146   |p_seq| == n &&
147   (\forall forall int i; 0 <= i && i < n; {p_seq[i]:} == p[i]) &&
148   (\forall forall int i; 0 <= i && i < n; BOTTOM() <= {p_seq[i]:} && p_seq[i] < n) &&
149   sum_p == prefixSum(p_seq, n, n);

```

Listing 8.3: pSeqInv: keeping the running total tied to p

VerCors first checks whether the first number decreases, and only if it stays the same does it check the second. The second number, $|\text{open_queue}| - \text{ptr}$, shrinks as nodes are taken off the queue within the round. The first number depends on sum_p , a ghost variable representing the running total by which every entry of p has grown so far, the budget from the intuition above. If a new node is added back to open_queue , which would otherwise make the second number grow, this can only happen because some entry of p just increased, which grows sum_p and therefore shrinks the first number by at least as much. So on every iteration, at least one of the two numbers strictly decreases.

For this to be a valid measure, two facts about sum_p must be proven: it can never exceed $\text{num_nodes} * (\text{num_nodes} + 1)$, so the first number cannot decrease forever, and sum_p really is the sum of how much every entry of p has grown, not merely a quantity that happens to satisfy that bound. VerCors has no built-in operator for summing the entries of an array or sequence, so both facts, and the notion of a running sum itself, are built from scratch.

The running total is defined by the pure function `prefixSum`, shown in Listing 8.2.

The sum over the first k entries of a sequence xs is the sum over the first $k-1$ entries, plus entry $k-1$ shifted so that `BOTTOM()` contributes 0 rather than a negative number. This recursive definition gives the bound in the second `ensures` clause of Listing 8.2 essentially for free: each of the k terms contributes at most $n - \text{BOTTOM}() - 1$ by construction, so the whole sum is bounded by k times that amount. Applied to the full array, with $k = n = \text{num_nodes}$ and `BOTTOM()` equal to -2 , this instantiates to exactly $\text{num_nodes} * (\text{num_nodes} + 1)$, the bound needed for the measure in Listing 8.1.

It is not enough for sum_p to merely satisfy this bound, it must genuinely equal the sum of p 's current entries. This is enforced by the predicate `pSeqInv`, shown in Listing 8.3, threaded through `processRoots`, `processNode`, and `resetActive` as a postcondition and loop invariant.

`p_seq` is a ghost mirror of p as a plain sequence. Listing 8.3 states that `p_seq` always matches p value for value, stays within the bounds p itself is constrained to, and that sum_p literally equals `prefixSum` of it, not merely a bound on it.

Every time an entry of p is written, `pSeqInv` has to be re-established: whatever sum_p becomes afterwards must again equal `prefixSum` of the new contents of p . This is what `lemma_prefixSum_update`, shown in Listing 8.4, proves.

To a human, the claim is intuitive: change one entry of a sequence from an old value to a new one, and the sum over the first k entries either stays the same, if the changed entry lies past position k , or shifts by exactly the change in that entry's value, if it lies within the first k entries. The difficulty is that `prefixSum` is defined recursively, and VerCors does not unfold a recursive definition by more than one layer on its own. Listing 8.4 supplies this reasoning

```

19 ensures k <= idx ==> prefixSum(xs[idx -> v], k, n) == prefixSum(xs, k, n);
20 ensures k > idx ==> prefixSum(xs[idx -> v], k, n) == prefixSum(xs, k, n) - xs[idx] + v;
21 decreases k;
22 void lemma_prefixSum_update(seq<int> xs, int idx, int v, int k, int n) {
23     if (k == 0) {
24     } else {
25         lemma_prefixSum_update(xs, idx, v, k - 1, n);
26     }
27 }

```

Listing 8.4: Proving that overwriting one entry shifts the sum by exactly that entry's change

```

29 requires n >= 0;
30 ensures |\result| == n;
31 ensures (\forall int i; 0 <= i && i < n; {:\result[i]:} == BOTTOM());
32 decreases n;
33 pure seq<int> allBottomSeq(int n) =
34     n == 0 ? seq<int>{} : allBottomSeq(n - 1) + seq<int>{BOTTOM()};
35
36 requires 0 <= k && k <= n;
37 requires |seq_x| == n;
38 requires (\forall int i; 0 <= i && i < n; {:\seq_x[i]:} == BOTTOM());
39 ensures prefixSum(seq_x, k, n) == 0;
40 decreases k;
41 void lemma_allBottom_sum_zero(seq<int> seq_x, int k, int n) {
42     if (k == 0) {
43     } else {
44         lemma_allBottom_sum_zero(seq_x, k - 1, n);
45     }
46 }

```

Listing 8.6: The all-unvisited sequence and its sum

as an explicit proof by induction on k , mirrored in the recursive shape of the method itself: at $k == 0$ both ensures clauses are trivial, and for $k > 0$, the recursive call grants the caller the already-proven ensures clauses for $k - 1$, the induction hypothesis, from which VerCors only needs to unfold `prefixSum` once to check the claim for k .

`lemma_prefixSum_update` is called every time an entry of p is written, illustrated here for `processRoots` in Listing 8.5.

```

238 ghost seq<int> p_seq_before = p_seq_out;
239 ghost p_seq_out = p_seq_out[r -> new_pr];
240 lemma_prefixSum_update(p_seq_before, r, new_pr, num_nodes, num_nodes);
241 ghost sum_p_out = sum_p_out + (new_pr - old_pr);

```

Listing 8.5: Calling `lemma_prefixSum_update` alongside a write to p

A snapshot of `p_seq_out` is kept before it is updated, and Listing 8.5 calls `lemma_prefixSum_update` on that snapshot with the index and new value just written, letting VerCors conclude how `prefixSum` changed, after which `sum_p_out` is updated by the same amount.

Finally, before any node has been processed, every entry of p is `BOTTOM()`, and `sum_p` needs to correctly start at 0 for `pSeqInv` to hold from the first line of `processRoots` onwards.

`allBottomSeq(n)` in Listing 8.6 builds the sequence of n entries all equal to `BOTTOM()`, matching the state of p before any node has been reached. `lemma_allBottom_sum_zero` proves, by the same style of induction on k , that `prefixSum` of any such sequence is exactly 0. Both functions are used together, once, at the very start of `processRoots`, shown in Listing 8.7.

```

208 ghost p_seq_out = allBottomSeq(num_nodes);
209 lemma_allBottom_sum_zero(p_seq_out, num_nodes, num_nodes);
210 ghost sum_p_out = 0;

```

Listing 8.7: Establishing the starting point of the sum

With Listing 8.3 established at the start of `processRoots` by Listing 8.7, and maintained across every write to `p` by Listing 8.4, `sum_p` is proven to genuinely track the total growth of `p`, and to be bounded by `num_nodes * (num_nodes + 1)`. This is exactly what Listing 8.1 needs, giving VerCors everything required to accept the measure without any `assume`, and completing the proof of Lemma 1.

8.2 Simple bounded loops

Four loops in the implementation, besides the round loop covered by Lemma 1 and the loop bounding the number of rounds in Section 8.3, each walk over a fixed range once, from an initial value up to a known bound, without ever revisiting an earlier position.

```

179 decreases n - chk;

```

Listing 8.8: Termination measure for the scan inside `anyInterrupted`

Listing 8.8 is the measure for the scan inside `anyInterrupted`, which checks whether a round produced any interruptions, and decreases as `chk` counts up towards `n`.

```

218 decreases num_roots - ri;

```

Listing 8.9: Termination measure for the seeding loop inside `processRoots`

Listing 8.9 is the measure for the seeding loop inside `processRoots`, which assigns an initial `p`-value to every root, and decreases as `ri` counts up towards `num_roots`.

```

303 decreases num_nodes - v;

```

Listing 8.10: Termination measure for the post-processing loop inside `resetActive`

Listing 8.10 is the measure for the post-processing loop inside `resetActive`, which re-opens or retires every node after a round, and decreases as `v` counts up towards `num_nodes`.

```

388 decreases degree(R, node) - ci;

```

Listing 8.11: Termination measure for the child loop inside `processNode`

Listing 8.11 is the measure for the child loop inside `processNode`, which inspects every child of the node currently being processed, and decreases as `ci` counts up towards `degree(R, node)`.

In each case, the loop variable starts at 0 and is incremented by exactly 1 on every iteration, while the loop's own guard keeps it below a fixed bound that does not change during the loop. This directly gives a strictly decreasing, non-negative measure, and VerCors accepts each of these four decreases clauses without needing any further invariants beyond the loop's own bounds.

8.3 Bounding the number of rounds

The more involved termination argument concerns the outer loop in `hitchhiking`, which runs once per round and stops once the open queue is empty and no interruptions remain. Its decreases clause, shown in Listing 8.12, uses the ghost variable `max_active_end_round`, already introduced in Section 7.5.

```

158 inline pure bool maxActiveInv(int num_nodes, int[] p, bool reset_happened, int
    max_active,
159                               int max_active_after_reset, int max_active_end_round,
160                               bool more_to_process) =
161   (reset_happened ==> (\forallall int v; 0 <= v && v < num_nodes; {p[v]:} != BOTTOM()))
    &&
162   (reset_happened ==> max_active == max_active_after_reset) &&
163   (more_to_process && reset_happened ==> max_active_after_reset < max_active_end_round
    ) &&
164   (more_to_process && !reset_happened ==> max_active_end_round == num_nodes);

```

Listing 8.13: maxActiveInv: relating the round-bounding ghost variables

```

511 decreases max_active_end_round - BOTTOM();

```

Listing 8.12: Termination measure for the outer round loop

The intuition behind this measure is that `max_active_end_round` cannot decrease indefinitely: it starts bounded above by `num_nodes` and is bounded below by `BOTTOM()`, so if it strictly decreases every round that continues, the number of rounds is bounded by `num_nodes` as well. This matches the informal argument in [14] that the algorithm cannot run for more rounds than there are nodes that can ever be activated, since post-processing retires the largest active node from consideration once its search is known to have failed to find a cycle, established by Lemma 5 in Section 7.5.

Making this precise in PVL needs two further ghost variables, in addition to `max_active_end_round`: `reset_happened`, a boolean that becomes true the first time `resetActive` is actually called, and never flips back, and `max_active_after_reset`, which snapshots the value of `max_active` immediately after such a reset. These are tied together with the loop's own measure by the predicate `maxActiveInv`, shown in Listing 8.13, one of the conjuncts of `processNodeInv` carried through the outer loop.

The last two conjuncts of Listing 8.13 are what matter here. The third states that once a reset has actually happened, the value of `max_active` recorded right after that reset is strictly smaller than the value of `max_active_end_round` recorded at the end of the following round, which is what makes the measure in Listing 8.12 actually decrease from one round to the next. The fourth covers the case before any reset has happened yet, where `max_active_end_round` is still at its placeholder value of `num_nodes`. Listing 8.13 is carried through the outer loop as a loop invariant, instantiated with the loop's own ghost variables, shown in Listing 8.14.

```

509 loop_invariant maxActiveInv(num_nodes, p, reset_happened, max_active,
    max_active_after_reset,
510                               max_active_end_round, ptr < |open_queue|);

```

Listing 8.14: maxActiveInv instantiated as the outer loop's invariant

With Listing 8.13 maintained throughout, `max_active_end_round` is proven to strictly decrease every round after the first one in which a reset occurs, and to stay at its fixed starting value `num_nodes` until that first reset happens, giving VerCors everything it needs to accept the measure in Listing 8.12 without any assume. Combined with Lemma 1 in Section 8.1, which guarantees every individual round terminates, and the simple loops of Section 8.2, this completes the proof that hitchhiking as a whole always terminates, whether or not an accepting cycle exists. Together with the partial correctness results of Chapters 6 and 7, this gives total correctness of hitchhiking.

Chapter 9

Conclusion and Future Work

9.1 An earlier attempt at Lemma 4

This section documents an earlier, unsuccessful attempt at Lemma 4, so a future attempt does not repeat the same mistake: proving an invariant that was not general enough, one that only captured a fact about the algorithm’s own bookkeeping rather than a fact about the graph itself.

Before Lemma 4 was set aside as future work (Section 7.4), an earlier attempt tried to prove it backward: instead of showing that the largest active node’s search finds a cycle when one is there, show that as long as the algorithm has not reported a cycle, that node cannot be on one. This looked more approachable, since it only asks whether something has *not* happened yet, rather than predicting what will. The attempt was built around tracking, for every node, how the algorithm had reached it so far, together with an invariant guaranteeing that this tracked information never showed a cycle through the largest active node. Maintaining this invariant in practice took a fair amount of extra bookkeeping throughout the algorithm.

This did not work. The invariant only described what the algorithm’s own bookkeeping happened to reflect at a given moment, not what was actually true of the graph: a cycle could exist through the largest active node long before exploration reached it, without ever violating the invariant, since nothing about it broke until the algorithm had already found the cycle, at which point the property was no longer useful. What Lemma 4 needs instead is a statement about the graph itself, true regardless of how far exploration has progressed, of the kind already sketched in Section 7.4.

9.2 Design choices made with future parallel verification in mind

Two design decisions in the current implementation, discussed in Chapter 5, were made to keep a future proof of the parallel, GPU-based version of Hitchhiking within reach, rather than purely to suit the sequential proof presented in this thesis. The algorithm’s own state, `p`, `in_open`, `in_active`, and `in_interrupted`, is represented as `int[]` arrays with explicit `Perm` permissions rather than as pure `seq<int>` sequences, even though the sequential algorithm never actually runs multiple threads over them: only heap arrays with permissions can be split across threads inside a `par` block, so this representation avoids the state arrays needing to be redesigned before that proof is attempted. Likewise, the graph is represented in Compressed Sparse Row format for its parallel-friendly access pattern, even though `R` and `C` would themselves still need to become `int[]` arrays for a parallel proof.

9.3 Where this thesis stops

This thesis set out to formally verify the sequential Hitchhiking algorithm using VerCors, addressing both research questions posed in the introduction. For RQ1, soundness of the sequential algorithm is fully proven, with no assume statements anywhere in the proof (Chapter 6). For RQ2, completeness is proven in part: Lemmas 1, 2, 3, and 5 of the six-lemma structure from [14] are fully verified, and termination of the algorithm, both per round and across all rounds, is fully proven (Chapter 8). Lemma 4, and the dependent Lemma 6, remain open, after the proof strategy attempted in Section 9.1 did not succeed. The parallel, GPU-based version of the algorithm was not attempted in this thesis, beyond the design choices discussed in Section 9.2.

9.4 Future work

Completing Lemma 4, and with it Lemma 6, is the most direct continuation of this work. Doing so will likely require formalising a genuine reachability predicate over the graph, such as the `pathExists` predicate sketched in Section 7.4.

Beyond completeness, the natural next step is the parallel, GPU-based version of Hitchhiking itself, the long-term goal behind the design choices of Section 9.2. This involves moving the CSR graph and the state arrays into a form usable inside a `par` block, and proving data-race freedom for the concurrent updates to `p` that the parallel algorithm in [14] performs atomically.

Bibliography

- [1] Lukas Armbrorst, Pieter Bos, Lars B van den Haak, Marieke Huisman, Robert Rubbens, Ömer Şakar, and Philip Tasche. The vercors verifier: a progress report. In *International Conference on Computer Aided Verification*, pages 3–18. Springer, 2024. 2, 35
- [2] Jiří Barnat, Petr Bauch, Luboš Brim, and Milan Češka. Designing fast ltl model checking algorithms for many-core gpus. *Journal of Parallel and Distributed Computing*, 72(9):1083–1097, 2012. 9, 11
- [3] Jiří Barnat, Luboš Brim, and Petr Ročkal. A time-optimal on-the-fly parallel algorithm for model checking of weak ltl properties. In *International Conference on Formal Engineering Methods*, pages 407–425. Springer, 2009. 1
- [4] Vincent Bloemen and Jaco van de Pol. Multi-core scc-based ltl model checking. In *Haifa Verification Conference*, pages 18–33. Springer, 2016. 1
- [5] Stefan Blom, Saeed Darabi, Marieke Huisman, and Wytse Oortwijn. The vercors tool set: verification of parallel and concurrent software. In *International Conference on Integrated Formal Methods*, pages 102–110. Springer, 2017. iii
- [6] Luboš Brim, Ivana Černá, Pavel Moravec, and Jiří Šimša. Accepting predecessors are better than back edges in distributed ltl model-checking. In *International Conference on Formal Methods in Computer-Aided Design*, pages 352–366. Springer, 2004. 1, 9, 11
- [7] Ivana Černá and Radek Pelánek. Distributed explicit fair cycle detection (set based approach). In *International SPIN Workshop on Model Checking of Software*, pages 49–73. Springer, 2003. 1
- [8] Costas Courcoubetis, Moshe Vardi, Pierre Wolper, and Mihalís Yannakakis. Memory-efficient algorithms for the verification of temporal properties. *Formal methods in system design*, 1(2):275–288, 1992. iii
- [9] Christine H Hoek. A small flaw translates into many miscalculations: A narrative criticism of the intel pentium chip crisis. 2002. 1
- [10] J. P. Hollander. Verification of a model checking algorithm in vercors. Master’s thesis, University of Twente, Enschede, The Netherlands, August 2021. 25
- [11] Gerard J Holzmann. Parallelizing the spin model checker. In *International SPIN Workshop on Model Checking of Software*, pages 155–171. Springer, 2012. 1
- [12] Marieke Huisman and Anton Wijs. *Concise Guide to Software Verification*. Springer, 2023. 1, 5, 7
- [13] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable gpu graph traversal. *ACM Sigplan Notices*, 47(8):117–128, 2012. 31, 35, 36

- [14] Muhammad Osama and Anton Wijs. Hitching a ride to a lasso: massively parallel on-the-fly ltl model checking. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 23–43. Springer, 2024. iii, 1, 5, 11, 19, 39, 41, 47, 48, 49, 50, 51, 52, 53, 55, 59, 62, 65
- [15] L. B. van den Haak. *Verifying Optimised Parallel Code*. PhD thesis, Eindhoven University of Technology, 2026. Unpublished doctoral thesis. 2, 25
- [16] Anton Wijs. Bfs-based model checking of linear-time properties with an application on gpus. In *International Conference on Computer Aided Verification*, pages 472–493. Springer, 2016. 1

Appendix A

Pseudocode of the Hitchhiking Algorithm

For reference, this appendix reproduces Algorithm 2 (Hitchhiking) from [14], made available under a Creative Commons Attribution 4.0 International License. The line numbers below match the original exactly, so that references to specific lines throughout this thesis can be checked directly against this listing.

Algorithm 1 The veri Hitchhiking algorithm, reproduced from [14].

```

1: function void Hitchhiking( $\mathcal{Q}_{0,\otimes}$ ,  $\text{succ}_{\otimes}()$ ):
2:    $\mathcal{O} \leftarrow \emptyset$                                  $\triangleright$  The open set, containing states to be explored
3:    $\mathcal{F} \leftarrow \emptyset$                              $\triangleright$  Set to keep track of cycle detection restarts
4:    $\mathcal{A} \leftarrow \emptyset$                              $\triangleright$  Set of accepting states for which cycle detection is ongoing
5:   forall  $\bar{q} \in \mathcal{Q}_{0,\otimes}$  do
6:      $\mathcal{O} \leftarrow \mathcal{O} \cup \{\bar{q}\}$ 
7:     if  $\bar{q} \in \mathcal{Q}_{F,\otimes}$  then  $\mathcal{A} \leftarrow \mathcal{A} \cup \{\bar{q}\}$ ;  $p(\bar{q}) \leftarrow \bar{q}$                                  $\triangleright \bar{q}$  is cycle active
8:     else  $p(\bar{q}) \leftarrow \epsilon$ 
9:     while  $\mathcal{O} \neq \emptyset$  do                                 $\triangleright$  Beginning of a round
10:      forall  $\bar{q} \in \mathcal{O}$  do in parallel
11:         $\mathcal{O} \leftarrow \mathcal{O} \setminus \{\bar{q}\}$ ;  $\alpha \leftarrow p(\bar{q})$ 
12:        forall  $\bar{q}' \in \text{succ}_{\otimes}(\bar{q})$  do in parallel
13:          if  $\alpha = \bar{q}'$  then return counter-example found
14:          if  $p(\bar{q}') = \perp$  and  $\bar{q}' \in \mathcal{Q}_{F,\otimes}$  then
15:             $\mathcal{A} \leftarrow \mathcal{A} \cup \{\bar{q}'\}$ 
16:            if  $\bar{q}' > \alpha$  then  $\alpha \leftarrow \bar{q}'$                                  $\triangleright$  Interrupted by  $\bar{q}' \in \mathcal{A}$ 
17:            else  $\mathcal{F} \leftarrow \mathcal{F} \cup \{\bar{q}'\}$ 
18:             $[\beta \leftarrow p(\bar{q}'); p(\bar{q}') \leftarrow \max(\alpha, \beta)]$                                  $\triangleright$  Atomic update of  $p(\bar{q}')$ 
19:            if  $\alpha > \beta$  and  $\beta > \epsilon$  and  $\bar{q}' \in \mathcal{O}$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{\beta\}$ 
20:            if  $\beta > \alpha$  and  $\alpha > \epsilon$  and  $\bar{q}' \notin \mathcal{A}$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{\alpha\}$ 
21:            if  $\alpha > \beta$  then  $\mathcal{O} \leftarrow \mathcal{O} \cup \{\bar{q}'\}$ 
22:      if  $\mathcal{F} \neq \emptyset$  then                                 $\triangleright$  Post-processing of a round
23:        forall  $\bar{q} \in \mathcal{Q}_{\otimes}$  do in parallel
24:          if  $\bar{q} \in \mathcal{A}$  then
25:            if  $\bar{q} \in \mathcal{F}$  and  $p(\bar{q}) \neq \bar{q}$  then  $\mathcal{O} \leftarrow \mathcal{O} \cup \{\bar{q}\}$ 
26:            else  $\mathcal{A} \leftarrow \mathcal{A} \setminus \{\bar{q}\}$ 
27:          if  $\bar{q} \in \mathcal{F}$  then  $\mathcal{F} \leftarrow \mathcal{F} \setminus \{\bar{q}\}$ 
28:          if  $\bar{q} \in \mathcal{A}$  then  $p(\bar{q}) \leftarrow \bar{q}$  else  $p(\bar{q}) \leftarrow \epsilon$ 
29:      return no counter-example found

```

Appendix B

The Verification Code of the Hitchhiking Algorithm

This appendix contains the complete PVL implementation of the sequential Hitchhiking algorithm discussed in Chapter 5, including the memory safety specification, the soundness proof from Chapter 6, and the completeness effort from Chapter 7. The file is verified automatically by VerCors.

```
1 pure int imax(int a, int b) = a > b ? a : b;
2 decreases;
3 pure int BOTTOM() = -2;
4 decreases;
5 pure int EPSILON() = -1;
6
7 requires 0 <= k && k <= |xs|;
8 requires (\forall int i; 0 <= i && i < |xs|; BOTTOM() <= {xs[i]:} && xs[i] < n);
9 ensures 0 <= \result;
10 ensures \result <= k * (n - BOTTOM() - 1);
11 decreases k;
12 pure int prefixSum(seq<int> xs, int k, int n) =
13   k == 0 ? 0 : prefixSum(xs, k - 1, n) + (xs[k - 1] - BOTTOM());
14
15 requires 0 <= idx && idx < |xs|;
16 requires 0 <= k && k <= |xs|;
17 requires BOTTOM() <= v && v < n;
18 requires (\forall int i; 0 <= i && i < |xs|; BOTTOM() <= {xs[i]:} && xs[i] < n);
19 ensures k <= idx ==> prefixSum(xs[idx -> v], k, n) == prefixSum(xs, k, n);
20 ensures k > idx ==> prefixSum(xs[idx -> v], k, n) == prefixSum(xs, k, n) - xs[idx] + v;
21 decreases k;
22 void lemma_prefixSum_update(seq<int> xs, int idx, int v, int k, int n) {
23   if (k == 0) {
24     } else {
25       lemma_prefixSum_update(xs, idx, v, k - 1, n);
26     }
27 }
28
29 requires n >= 0;
30 ensures |\result| == n;
31 ensures (\forall int i; 0 <= i && i < n; {:\result[i]:} == BOTTOM());
32 decreases n;
33 pure seq<int> allBottomSeq(int n) =
34   n == 0 ? seq<int>{} : allBottomSeq(n - 1) + seq<int>{BOTTOM()};
35
36 requires 0 <= k && k <= n;
37 requires |seq_x| == n;
38 requires (\forall int i; 0 <= i && i < n; {seq_x[i]:} == BOTTOM());
39 ensures prefixSum(seq_x, k, n) == 0;
40 decreases k;
```

```

41 void lemma_allBottom_sum_zero(seq<int> seq_x, int k, int n) {
42   if (k == 0) {
43     } else {
44       lemma_allBottom_sum_zero(seq_x, k - 1, n);
45     }
46 }
47
48 requires 0 <= u && u < |R| - 1;
49 requires 0 <= R[u] && R[u+1] <= |C|;
50 pure bool isEdge(seq<int> R, seq<int> C, int u, int v) =
51   (\exists int j; R[u] <= j && j < R[u+1]; {C[j]:} == v);
52
53 // number of successors of node u in CSR
54 inline pure int degree(seq<int> R, int u) = R[u+1] - R[u];
55
56 inline pure bool validGraph(int N, seq<int> R, seq<int> C) =
57   |R| == N + 1 &&
58   R[0] == 0 &&
59   |C| == R[N] &&
60   (\forall int u; 0 <= u && u < N; {R[u]:} <= R[u+1]) &&
61   (\forall int u; 0 <= u && u < N; 0 <= {R[u]:} && R[u] <= |C|) &&
62   (\forall int i; 0 <= i && i < |C|; 0 <= {C[i]:} && C[i] < N);
63
64 requires validGraph(N, R, C);
65 inline pure bool isAPath(int N, seq<int> R, seq<int> C, seq<int> path) =
66   |path| > 0 &&
67   (\forall int i; 0 <= i && i < |path|; 0 <= {path[i]:} && path[i] < N) &&
68   (\forall int i; 0 <= i && i < |path| - 1; isEdge(R, C, {path[i]:}, path[i+1]));
69
70 requires validGraph(N, R, C);
71 requires |is_red| == N;
72 inline pure bool isAcceptingCycle(int N, seq<int> R, seq<int> C, seq<int> is_red, seq<
73   int> path) =
74   isAPath(N, R, C, path) &&
75   isEdge(R, C, path[|path| - 1], path[0]) &&
76   (\exists int i; 0 <= i && i < |path|; is_red[{path[i]:}] == 1);
77
78 inline pure bool arr(int[] a, int n) = a != null && a.length == n;
79
80 inline pure bool validAlgoArrays(int[] p, int[] in_open, int[] in_active, int[]
81   in_interrupted, int n) =
82   arr(p, n) && arr(in_open, n) && arr(in_active, n) && arr(in_interrupted, n);
83
84 inline pure bool allEq(int[] a, int n, int val) =
85   (\forall int v; 0 <= v && v < n; {a[v]:} == val);
86
87 inline pure bool validInput(int num_nodes, seq<int> is_red, seq<int> R, seq<int> C) =
88   num_nodes > 0 &&
89   |is_red| == num_nodes &&
90   validGraph(num_nodes, R, C) &&
91   (\forall int v; 0 <= v && v < num_nodes; {is_red[v]:} == 0 || is_red[v] == 1);
92
93 inline pure bool validRoots(int num_roots, seq<int> roots, int num_nodes) =
94   num_roots > 0 && |roots| == num_roots &&
95   (\forall int i; 0 <= i && i < num_roots; 0 <= {roots[i]:} && roots[i] < num_nodes);
96
97 inline pure bool flags01(int[] a, int n) =
98   (\forall int v; 0 <= v && v < n; {a[v]:} == 0 || a[v] == 1);
99
100 inline pure bool pWellFormed(int[] p, seq<int> is_red, int n) =
101   (\forall int v; 0 <= v && v < n;
102     {p[v]:} == BOTTOM() || p[v] == EPSILON() || (0 <= p[v] && p[v] < n)) &&
103   (\forall int v; 0 <= v && v < n;
104     (0 <= {p[v]:} && p[v] < n) ==> is_red[p[v]] == 1);
105
106 inline pure bool activeImpliesRed(int[] in_active, seq<int> is_red, int n) =
107   (\forall int v; 0 <= v && v < n; {in_active[v]:} == 1 ==> is_red[v] == 1);

```



```

106
107 inline pure bool queueOk(seq<int> q, int[] p, int n) =
108   (\forallall int j; 0 <= j && j < |q|; 0 <= {q[j]:} && q[j] < n) &&
109   (\forallall int j; 0 <= j && j < |q|; {p[q[j]]:} != BOTTOM());
110
111 inline pure bool ghostPathsOk(int n, seq<int> R, seq<int> C, seq<int>[] ghost_path, int
112   [] p) =
113   (0 <= {p[v]:} && p[v] < n) ==>
114   |ghost_path[v]| >= 1 &&
115   ghost_path[v][0] == p[v] &&
116   ghost_path[v][|ghost_path[v]| - 1] == v &&
117   (\forallall int i; 0 <= i && i < |ghost_path[v]|;
118     0 <= {ghost_path[v][i]:} && ghost_path[v][i] < n) &&
119   (\forallall int i; 0 <= i && i < |ghost_path[v]| - 1;
120     isEdge(R, C, {ghost_path[v][i]:}, ghost_path[v][i+1]));
121
122 inline pure bool maxActiveOk(int[] p, int[] in_active, int m, int n) =
123   (m == BOTTOM() || (0 <= m && m < n)) &&
124   (\forallall int w; 0 <= w && w < n; {p[w]:} >= 0 ==> p[w] <= m) &&
125   (\forallall int v; 0 <= v && v < n; {in_active[v]:} == 1 ==> v <= m);
126
127 inline pure bool stateInv(int n, seq<int> R, seq<int> C, seq<int> is_red, seq<int> q,
128   int[] p, int[] in_open, int[] in_active, seq<int>[] ghost_path
129   , int m) =
130   flags01(in_open, n) &&
131   flags01(in_active, n) &&
132   queueOk(q, p, n) &&
133   pWellFormed(p, is_red, n) &&
134   activeImpliesRed(in_active, is_red, n) &&
135   ghostPathsOk(n, R, C, ghost_path, p) &&
136   maxActiveOk(p, in_active, m, n);
137
138 requires m == BOTTOM() || (0 <= m && m < n);
139 inline pure bool interruptionInv(int n, int[] p, int[] in_active, int[] in_interrupted,
140   int m) =
141   flags01(in_interrupted, n) &&
142   (\forallall int i; 0 <= i && i < n; {in_interrupted[i]:} == 1 ==> in_active[i] == 1)
143   &&
144   (\forallall int w; 0 <= w && w < n; 0 <= {p[w]:} && p[w] < n ==> in_active[p[w]] == 1)
145   &&
146   (m == BOTTOM() ==> (\forallall int i; 0 <= i && i < n; {in_interrupted[i]:} == 0)) &&
147   (m == BOTTOM() || in_interrupted[m] == 0);
148
149 inline pure bool pSeqInv(int n, int[] p, seq<int> p_seq, int sum_p) =
150   |p_seq| == n &&
151   (\forallall int i; 0 <= i && i < n; {p_seq[i]:} == p[i]) &&
152   (\forallall int i; 0 <= i && i < n; BOTTOM() <= {p_seq[i]:} && p_seq[i] < n) &&
153   sum_p == prefixSum(p_seq, n, n);
154
155 inline pure bool processNodeInv(int n, seq<int> R, seq<int> C, seq<int> is_red, seq<int>
156   q,
157   int[] p, int[] in_open, int[] in_active, seq<int>[]
158   ghost_path,
159   int[] in_interrupted, int m, seq<int> p_seq, int sum_p)
160   =
161   stateInv(n, R, C, is_red, q, p, in_open, in_active, ghost_path, m) &&
162   interruptionInv(n, p, in_active, in_interrupted, m) &&
163   pSeqInv(n, p, p_seq, sum_p);
164
165 inline pure bool maxActiveInv(int num_nodes, int[] p, bool reset_happened, int
166   max_active,
167   int max_active_after_reset, int max_active_end_round,
168   bool more_to_process) =
169   (reset_happened ==> (\forallall int v; 0 <= v && v < num_nodes; {p[v]:} != BOTTOM()))
170   &&
171   (reset_happened ==> max_active == max_active_after_reset) &&

```

```

163     (more_to_process && reset_happened ==> max_active_after_reset < max_active_end_round
164     ) &&
165     (more_to_process && !reset_happened ==> max_active_end_round == num_nodes);
166 // check if there are interrupted nodes
167 requires n >= 0;
168 context in_interrupted != null && in_interrupted.length == n;
169 context Perm(in_interrupted[*], write);
170 ensures (\forall forall int i; 0 <= i && i < n; {in_interrupted[i]:} == \old(in_interrupted[i]
171 ));
172 ensures \result == (\exists int i; 0 <= i && i < n; {in_interrupted[i]:} == 1);
173 bool anyInterrupted(int n, int[] in_interrupted) {
174     bool found_any = false;
175     int chk = 0;
176     loop_invariant Perm(in_interrupted[*], write);
177     loop_invariant 0 <= chk && chk <= n;
178     loop_invariant (\forall forall int i; 0 <= i && i < n; {in_interrupted[i]:} == \old(
179     in_interrupted[i]));
180     loop_invariant found_any == (\exists int i; 0 <= i && i < chk; {in_interrupted[i]:}
181     == 1);
182     decreases n - chk;
183     while (chk < n) {
184         if (in_interrupted[chk] == 1) { found_any = true; }
185         chk = chk + 1;
186     }
187     return found_any;
188 }
189 // process the roots, returns the queue of open nodes
190 context_everywhere validInput(num_nodes, is_red, R, C);
191 context_everywhere validRoots(num_roots, roots, num_nodes);
192 context_everywhere arr(p, num_nodes) && arr(in_open, num_nodes) && arr(in_active,
193 num_nodes);
194 context_everywhere ghost_path != null && ghost_path.length == num_nodes;
195 context_everywhere Perm(p[*], write);
196 context_everywhere Perm(in_open[*], write);
197 context_everywhere Perm(in_active[*], write);
198 context_everywhere Perm(ghost_path[*], write);
199 requires allEq(p, num_nodes, BOTTOM()) && allEq(in_open, num_nodes, 0) && allEq(
200 in_active, num_nodes, 0);
201 yields int max_active_out;
202 yields seq<int> p_seq_out;
203 yields int sum_p_out;
204 ensures stateInv(num_nodes, R, C, is_red, \result, p, in_open, in_active, ghost_path,
205 max_active_out);
206 ensures pSeqInv(num_nodes, p, p_seq_out, sum_p_out);
207 ensures (\forall forall int w; 0 <= w && w < num_nodes; 0 <= {p[w]:} && p[w] < num_nodes ==>
208 in_active[p[w]] == 1);
209 seq<int> processRoots(int num_nodes, int num_roots, seq<int> roots, seq<int> is_red,
210 seq<int> R, seq<int> C,
211 int[] p, int[] in_open, int[] in_active, seq<int>[] ghost_path)
212 {
213     ghost max_active_out = BOTTOM();
214     ghost p_seq_out = allBottomSeq(num_nodes);
215     lemma_allBottom_sum_zero(p_seq_out, num_nodes, num_nodes);
216     ghost sum_p_out = 0;
217     seq<int> open_queue = seq<int>{};
218     int ri = 0;
219     loop_invariant 0 <= ri && ri <= num_roots;
220     loop_invariant |open_queue| <= ri;
221     loop_invariant stateInv(num_nodes, R, C, is_red, open_queue, p, in_open, in_active,
222     ghost_path, max_active_out);
223     loop_invariant pSeqInv(num_nodes, p, p_seq_out, sum_p_out);
224     loop_invariant (\forall forall int w; 0 <= w && w < num_nodes; 0 <= {p[w]:} && p[w] <
225     num_nodes ==> in_active[p[w]] == 1);
226     decreases num_roots - ri;
227     while (ri < num_roots) {

```

```

220     int r = roots[ri];
221     if (in_open[r] == 0) {
222         open_queue = open_queue + [r];
223         in_open[r] = 1;
224     }
225     int old_pr = p[r];
226     int new_pr;
227     if (is_red[r] == 1) {
228         in_active[r] = 1;
229         ghost if (r > max_active_out) { max_active_out = r; }
230         p[r] = r;
231         new_pr = r;
232         ghost ghost_path[r] = [r];
233     } else {
234         p[r] = EPSILON();
235         new_pr = EPSILON();
236         ghost ghost_path[r] = seq<int>{};
237     }
238     ghost seq<int> p_seq_before = p_seq_out;
239     ghost p_seq_out = p_seq_out[r → new_pr];
240     lemma_prefixSum_update(p_seq_before, r, new_pr, num_nodes, num_nodes);
241     ghost sum_p_out = sum_p_out + (new_pr - old_pr);
242     ri = ri + 1;
243 }
244 return open_queue;
245 }
246
247 // post-processing
248 context_everywhere validInput(num_nodes, is_red, R, C);
249 context_everywhere validAlgoArrays(p, in_open, in_active, in_interrupted, num_nodes);
250 context_everywhere ghost_path != null && ghost_path.length == num_nodes;
251 context_everywhere Perm(p[*], write);
252 context_everywhere Perm(in_open[*], write);
253 context_everywhere Perm(in_active[*], write);
254 context_everywhere Perm(in_interrupted[*], write);
255 context_everywhere Perm(ghost_path[*], write);
256 given int max_active_in;
257 yields int max_active_out;
258 requires flags01(in_interrupted, num_nodes);
259 requires stateInv(num_nodes, R, C, is_red, open_queue, p, in_open, in_active, ghost_path,
    , max_active_in);
260 requires max_active_in == BOTTOM() || in_interrupted[max_active_in] == 0;
261 requires (\forall int i; 0 <= i && i < num_nodes; {in_interrupted[i]:} == 1 ==>
    in_active[i] == 1);
262 ensures |\result| >= |open_queue|;
263 ensures flags01(in_interrupted, num_nodes);
264 ensures stateInv(num_nodes, R, C, is_red, \result, p, in_open, in_active, ghost_path,
    max_active_out);
265 ensures max_active_in == BOTTOM() || max_active_out < max_active_in;
266 ensures (\forall int v; 0 <= v && v < num_nodes; {p[v]:} != BOTTOM());
267 ensures (\forall int i; 0 <= i && i < num_nodes; {in_interrupted[i]:} == 1 ==>
    in_active[i] == 1);
268 ensures (\forall int w; 0 <= w && w < num_nodes; 0 <= {p[w]:} && p[w] < num_nodes ==>
    in_active[p[w]] == 1);
269 ensures (\forall int i; 0 <= i && i < num_nodes; {in_interrupted[i]:} == 0);
270 given seq<int> p_seq_in;
271 given int sum_p_in;
272 yields seq<int> p_seq_out;
273 yields int sum_p_out;
274 requires pSeqInv(num_nodes, p, p_seq_in, sum_p_in);
275 ensures pSeqInv(num_nodes, p, p_seq_out, sum_p_out);
276 seq<int> resetActive(int num_nodes, seq<int> is_red,
277     seq<int> R, seq<int> C, seq<int> open_queue,
278     int[] p, int[] in_open, int[] in_active, int[] in_interrupted,
279     seq<int>[] ghost_path)
280 {
281     ghost max_active_out = BOTTOM();

```

```

282 ghost p_seq_out = p_seq_in;
283 ghost sum_p_out = sum_p_in;
284 seq<int> q = open_queue;
285 int v = 0;
286 loop_invariant 0 <= v && v <= num_nodes;
287 loop_invariant |q| >= |open_queue|;
288 loop_invariant flags01(in_interrupted, num_nodes);
289 loop_invariant (\forall forall int w; 0 <= w && w < v; {p[w]:} != BOTTOM());
290 loop_invariant (\forall forall int i; 0 <= i && i < num_nodes; {in_interrupted[i]:} == 1
    ==> in_active[i] == 1);
291 loop_invariant (\forall forall int w; 0 <= w && w < v; 0 <= {p[w]:} && p[w] < num_nodes
    ==> in_active[p[w]] == 1);
292 loop_invariant (\forall forall int i; 0 <= i && i < v; {in_interrupted[i]:} == 0);
293 loop_invariant stateInv(num_nodes, R, C, is_red, q, p, in_open, in_active,
    ghost_path, max_active_in);
294 loop_invariant max_active_out == BOTTOM() || (0 <= max_active_out && max_active_out
    < num_nodes);
295 loop_invariant (\forall forall int w; 0 <= w && w < v; {in_active[w]:} == 1 ==> w <=
    max_active_out);
296 loop_invariant (\forall forall int w; 0 <= w && w < v; {p[w]:} >= 0 ==> p[w] <=
    max_active_out);
297 loop_invariant max_active_out == BOTTOM() ||
298     (0 <= max_active_out && max_active_out < v && in_active[
        max_active_out] == 1);
299 loop_invariant max_active_in == BOTTOM() ||
300     (v <= max_active_in ==> in_interrupted[max_active_in] == 0);
301 loop_invariant max_active_in == BOTTOM() || v <= max_active_in || in_active[
    max_active_in] == 0;
302 loop_invariant pSeqInv(num_nodes, p, p_seq_out, sum_p_out);
303 decreases num_nodes - v;
304 while (v < num_nodes) {
305     if (in_active[v] == 1) {
306         if (in_interrupted[v] == 1 && p[v] != v) {
307             if (in_open[v] == 0) {
308                 q = q + [v];
309                 in_open[v] = 1;
310             }
311         } else {
312             in_active[v] = 0;
313         }
314         in_interrupted[v] = 0;
315     }
316     int old_pv = p[v];
317     int new_pv;
318     if (in_active[v] == 1) {
319         p[v] = v;
320         new_pv = v;
321         ghost if (v > max_active_out) { max_active_out = v; }
322         ghost ghost_path[v] = [v];
323     } else {
324         p[v] = EPSILON();
325         new_pv = EPSILON();
326         ghost ghost_path[v] = seq<int>{};
327     }
328     ghost seq<int> p_seq_before = p_seq_out;
329     ghost p_seq_out = p_seq_out[v -> new_pv];
330     lemma_prefixSum_update(p_seq_before, v, new_pv, num_nodes, num_nodes);
331     ghost sum_p_out = sum_p_out + (new_pv - old_pv);
332     v = v + 1;
333 }
334 return q;
335 }
336
337 // process the children of a node
338 context_everywhere validInput(num_nodes, is_red, R, C);
339 context_everywhere validAlgoArrays(p, in_open, in_active, in_interrupted, num_nodes);
340 context_everywhere ghost_path != null && ghost_path.length == num_nodes;

```

```

341 context_everywhere found != null && found.length == 1;
342 context_everywhere Perm(p[*], write);
343 context_everywhere Perm(in_open[*], write);
344 context_everywhere Perm(in_active[*], write);
345 context_everywhere Perm(in_interrupted[*], write);
346 context_everywhere Perm(ghost_path[*], write);
347 context_everywhere Perm(found[*], write);
348 given int sum_p_in;
349 given int max_active_in;
350 given seq<int> p_seq_in;
351 yields int sum_p_out;
352 yields int max_active_out;
353 yields seq<int> cycle_out;
354 yields seq<int> p_seq_out;
355 requires 0 <= node && node < num_nodes;
356 requires found[0] == 0;
357 requires processNodeInv(num_nodes, R, C, is_red, open_queue, p, in_open, in_active,
    ghost_path,
358                          in_interrupted, max_active_in, p_seq_in, sum_p_in);
359 ensures found[0] == 0 || found[0] == 1;
360 ensures |\\result| >= |open_queue|;
361 ensures processNodeInv(num_nodes, R, C, is_red, \\result, p, in_open, in_active,
    ghost_path,
362                          in_interrupted, max_active_out, p_seq_out, sum_p_out);
363 ensures sum_p_out >= sum_p_in;
364 ensures sum_p_out - sum_p_in >= |\\result| - |open_queue|;
365 ensures found[0] == 1 ==> isAcceptingCycle(num_nodes, R, C, is_red, cycle_out);
366 ensures (\\forall int w; 0 <= w && w < num_nodes; \\old(p[w]) != BOTTOM() ==> {p[w]:} !=
    BOTTOM());
367 ensures (\\forall int w; 0 <= w && w < num_nodes; \\old({p[w]:}) != BOTTOM() ==>
    max_active_out == max_active_in;
368 seq<int> processNode(int num_nodes, seq<int> is_red,
369                      seq<int> R, seq<int> C, int node, seq<int> open_queue,
370                      int[] p, int[] in_open, int[] in_active, int[] in_interrupted,
371                      int[] found, seq<int>[] ghost_path)
372 {
373     ghost sum_p_out = sum_p_in;
374     ghost max_active_out = max_active_in;
375     ghost p_seq_out = p_seq_in;
376     seq<int> q = open_queue;
377     int ci = 0;
378     loop_invariant found[0] == 0;
379     loop_invariant 0 <= ci && ci <= degree(R, node);
380     loop_invariant 0 <= R[node] && R[node] + degree(R, node) <= |C|;
381     loop_invariant |q| >= |open_queue|;
382     loop_invariant (\\forall int w; 0 <= w && w < num_nodes; \\old(p[w]) != BOTTOM() ==>
        {p[w]:} != BOTTOM());
383     loop_invariant (\\forall int w; 0 <= w && w < num_nodes; \\old({p[w]:}) != BOTTOM()
        ==> max_active_out == max_active_in;
384     loop_invariant processNodeInv(num_nodes, R, C, is_red, q, p, in_open, in_active,
        ghost_path,
385                          in_interrupted, max_active_out, p_seq_out, sum_p_out);
386     loop_invariant sum_p_out >= sum_p_in;
387     loop_invariant sum_p_out - sum_p_in >= |q| - |open_queue|;
388     decreases degree(R, node) - ci;
389     while (ci < degree(R, node)) {
390         int alpha = p[node];
391         int child = C[R[node] + ci];
392         assert isEdge(R, C, node, child);
393
394         // line 13: accepting cycle — exit early
395         if (alpha == child) {
396             found[0] = 1;
397             assert is_red[child] == 1;
398             assert ghost_path[node][0] == child;
399             assert ghost_path[node][|ghost_path[node]| - 1] == node;
400             ghost cycle_out = ghost_path[node];

```

```

401     assert |cycle_out| >= 1;
402     assert (\forall forall int i; 0 <= i && i < |cycle_out|;
403           0 <= { :cycle_out[i]: } && cycle_out[i] < num_nodes);
404     assert (\forall forall int i; 0 <= i && i < |cycle_out| - 1;
405           isEdge(R, C, { :cycle_out[i]: }, cycle_out[i+1]));
406     assert isEdge(R, C, cycle_out[|cycle_out| - 1], cycle_out[0]);
407     assert is_red[cycle_out[0]] == 1;
408     return q;
409 }
410
411 // lines 14–17: first visit to a red child
412 if (p[child] == BOTTOM() && is_red[child] == 1) {
413     in_active[child] = 1;
414     ghost if (child > max_active_out) { max_active_out = child; }
415     if (child > alpha) {
416         alpha = child;
417     } else {
418         in_interrupted[child] = 1;
419     }
420 }
421
422 // line 18: atomic max update of p[child]
423 int beta = p[child];
424 assert 0 <= child && child < num_nodes;
425 ghost seq<int> p_seq_before = p_seq_out;
426 p[child] = imax(alpha, beta);
427 ghost p_seq_out = p_seq_out[child → p[child]];
428 lemma_prefixSum_update(p_seq_before, child, p[child], num_nodes, num_nodes);
429 ghost if (alpha > beta) {
430     ghost sum_p_out = sum_p_out + alpha - beta;
431 }
432
433 // line 19: 2nd interruption
434 if (alpha > beta && beta > EPSILON() && in_open[child] == 1) {
435     in_interrupted[beta] = 1;
436 }
437
438 // line 20: 3rd interruption
439 if (beta > alpha && alpha > EPSILON() && in_active[child] == 0) {
440     in_interrupted[alpha] = 1;
441 }
442
443 // line 21: reopen child if p grew (deduplicated via in_open check)
444 if (alpha > beta && in_open[child] == 0) {
445     q = q + [child];
446     in_open[child] = 1;
447 }
448
449 ghost if (p[child] == child) {
450     ghost_path[child] = [child];
451 } else ghost if (p[child] == p[node] && p[child] >= 0) {
452     ghost_path[child] = ghost_path[node] + [child];
453 }
454
455 ci = ci + 1;
456 }
457 return q;
458 }
459
460 // check correct inputs, permissions, initial states
461 context everywhere validInput(num_nodes, is_red, R, C);
462 context everywhere validRoots(num_roots, roots, num_nodes);
463 context everywhere validAlgoArrays(p, in_open, in_active, in_interrupted, num_nodes);
464 context everywhere Perm(p[*], write);
465 context everywhere Perm(in_open[*], write);
466 context everywhere Perm(in_active[*], write);
467 context everywhere Perm(in_interrupted[*], write);

```

```

468 requires allEq(p, num_nodes, BOTTOM()) && allEq(in_open, num_nodes, 0) &&
469         allEq(in_active, num_nodes, 0) && allEq(in_interrupted, num_nodes, 0);
470
471 // soundness check: if returns true → there exists a cycle
472 yields seq<int> cycle;
473 ensures \result ==> isAcceptingCycle(num_nodes, R, C, is_red, cycle);
474
475 bool hitchhiking(
476     int num_nodes,
477     int num_roots,
478     seq<int> roots,
479     seq<int> is_red,
480     seq<int> R,
481     seq<int> C,
482     int[] p,
483     int[] in_open,
484     int[] in_active,
485     int[] in_interrupted
486 ) {
487     ghost seq<int>[] ghost_path = new seq<int>[num_nodes];
488     ghost int max_active = BOTTOM();
489     ghost seq<int> p_seq = allBottomSeq(num_nodes);
490     ghost int sum_p = 0;
491     int[] found = new int[1];
492     found[0] = 0;
493     int ptr = 0;
494
495     seq<int> open_queue = processRoots(num_nodes, num_roots, roots, is_red, R, C,
496                                     p, in_open, in_active, ghost_path)
497                               yields { max_active = max_active_out, p_seq = p_seq_out, sum_p
                                   = sum_p_out };
498
499     ghost int max_active_end_round = num_nodes;
500     ghost bool reset_happened = false;
501     ghost int max_active_after_reset = BOTTOM();
502
503     loop_invariant Perm(ghost_path[*], write);
504     loop_invariant Perm(found[*], write);
505     loop_invariant found[0] == 0;
506     loop_invariant 0 <= ptr && ptr <= |open_queue|;
507     loop_invariant processNodeInv(num_nodes, R, C, is_red, open_queue, p, in_open,
508                                in_active, ghost_path,
509                                in_interrupted, max_active, p_seq, sum_p);
510     loop_invariant maxActiveInv(num_nodes, p, reset_happened, max_active,
511                                max_active_after_reset,
512                                max_active_end_round, ptr < |open_queue|);
513     decreases max_active_end_round - BOTTOM();
514     while (ptr < |open_queue|) {
515
516         loop_invariant Perm(ghost_path[*], write);
517         loop_invariant Perm(found[*], write);
518         loop_invariant found[0] == 0;
519         loop_invariant 0 <= ptr && ptr <= |open_queue|;
520         loop_invariant processNodeInv(num_nodes, R, C, is_red, open_queue, p, in_open,
521                                    in_active, ghost_path,
522                                    in_interrupted, max_active, p_seq, sum_p);
523         loop_invariant maxActiveInv(num_nodes, p, reset_happened, max_active,
524                                    max_active_after_reset,
525                                    max_active_end_round, ptr < |open_queue|);
526         decreases num_nodes * (num_nodes + 1) - sum_p, |open_queue| - ptr;
527
528         while (ptr < |open_queue|) {
529             int node = open_queue[ptr];
530             ptr = ptr + 1;
531             in_open[node] = 0;
532
533             open_queue = processNode(num_nodes, is_red, R, C, node, open_queue,

```

```

530         p, in_open, in_active, in_interrupted, found,
531             ghost_path)
532     given { sum_p_in = sum_p, max_active_in = max_active, p_seq_in
533         = p_seq }
534     yields { sum_p = sum_p_out, max_active = max_active_out, cycle
535         = cycle_out, p_seq = p_seq_out };
536
537     if (found[0] == 1) {
538         return true; // a counter example found
539     }
540
541     max_active_end_round = max_active;
542
543     // Lemma 5
544     assert max_active_end_round == BOTTOM() || (0 <= max_active_end_round &&
545         max_active_end_round < num_nodes && in_interrupted[max_active_end_round] ==
546         0);
547
548     bool any_interrupted = anyInterrupted(num_nodes, in_interrupted);
549
550     // lines 23–28 in the pseudocode
551     if (any_interrupted) {
552         open_queue = resetActive(num_nodes, is_red, R, C, open_queue,
553             p, in_open, in_active, in_interrupted, ghost_path)
554         given { max_active_in = max_active, p_seq_in = p_seq, sum_p_in
555             = sum_p }
556         yields { max_active = max_active_out, p_seq = p_seq_out, sum_p
557             = sum_p_out };
558         ghost max_active_after_reset = max_active;
559         ghost reset_happened = true;
560     }
561
562     return false; // no counter example found
563 }

```

Listing B.1: The complete PVL implementation